Ensemble Network

Building Distributed AI Systems That Never Sleep

Matthew Dickinson

March 2026

Contents

Chapter 1: The Hotel That Never Sleeps **7**
The Problem Nobody Is Talking About . . . 7
Walk Into a Hotel . . . 7
The Architectural Blueprint . . . 8
What Current Frameworks Cannot Model . . . 9
Three Properties That Matter . . . 10
Property 1: Always-On . . . 10
Property 2: Decentralized . . . 10
Property 3: Human-Augmented, Not Human-Dependent . . . 10
What We Are Building . . . 10

Chapter 2: Ensembles as Services **12**
Scripts Versus Services . . . 12
The Long-Running Ensemble . . . 12
What an Ensemble Shares . . . 13
Shared Tasks . . . 13
Shared Tools . . . 13
Consuming Shared Capabilities . . . 14
Three Task Modes . . . 14
Shared (Reactive) . . . 14
Scheduled (Proactive) . . . 15
Internal (Private) . . . 15
The Network . . . 15
What Does Not Change . . . 16
From Script to Service: The Mental Model Shift . . . 16

Chapter 3: Cross-Ensemble Delegation **17**
Borrowing a Tool Versus Hiring a Department . . . 17
What Happens During a Cross-Ensemble Task Delegation . . . 17
The Delegation Chain . . . 19
Depth Limiting . . . 19
NetworkTool: The Lighter Alternative . . . 19
Transparent to the Agent . . . 20
When to Use Which . . . 21

Chapter 4: The WorkRequest Protocol **22**

A Work Order, Not a Function Call . . . 22
The Envelope . . . 22
"Call Me Back at This Number" . . . 23
Delivery Methods . . . 24
Ingress: How Work Arrives . . . 24
The Request-Response Cycle . . . 25
Why Natural Language Context? . . . 25

Chapter 5: Capacity Management – Bend, Don't Break **26**
The Instinct to Reject . . . 26
Accept, Queue, Estimate, Process, Deliver . . . 26
Caller-Side SLAs . . . 27
Priority Queuing . . . 27
Priority Aging . . . 28
Human Re-Prioritization . . . 28
Operational Profiles . . . 28
Elastic Scaling Within a Profile . . . 29
Hard Limits . . . 29
The Ticket Seller on the Street . . . 29

Chapter 6: Durable Transport **31**
The Fragility of Direct Connections . . . 31
Two Layers, One Protocol . . . 31
Real-Time Layer (WebSocket) . . . 31
Durable Layer (Message Queues + Result Store) . . . 32
The Asymmetric Routing Problem . . . 32
The Transport SPI . . . 32
Simple Transport (Development) . . . 32
Durable Transport (Production) . . . 33
Custom Transport . . . 33
Consumer Groups and Horizontal Scaling . . . 33
Visibility Timeout . . . 34
When WebSocket is Enough . . . 34
The State of the Art . . . 34

Chapter 7: Human Participation **36**
The Manager Who Goes Home . . . 36
The Interaction Spectrum . . . 36
Autonomous . . . 36
Advisory . . . 36
Notifiable . . . 37
Approvable . . . 37
Gated . . . 37
Out-of-Band Notification . . . 38
Directives: Non-Blocking Guidance . . . 38
Context Directives . . . 38
Control Plane Directives . . . 39
Who Can Send Directives? . . . 39

Late-Join: Catching Humans Up . . . 39
Role-Based Access . . . 40
The Dashboard as a Portal . . . 40

Chapter 8: Observability **42**
You Cannot Debug What You Cannot See . . . 42
Distributed Tracing: The Thread Through the Maze . . . 42
W3C Trace Context . . . 42
OpenTelemetry Integration . . . 43
ExecutionTrace Correlation . . . 43
Adaptive Audit Trail . . . 44
The Problem with Static Logging . . . 44
Leveled Auditing . . . 44
Dynamic Rules . . . 44
Audit Sinks . . . 45
Scaling Metrics . . . 46
Ensemble-Level Metrics . . . 46
Cross-Ensemble Metrics . . . 46
Token Cost Metrics . . . 46
The Observability Stack . . . 46

Chapter 9: Error Handling and Resilience **48**
Five Ways Things Go Wrong . . . 48
Failure Mode A: Unreachable . . . 48
Failure Mode B: Busy . . . 48
Failure Mode C: Crash Mid-Execution . . . 48
Failure Mode D: Business Error . . . 48
Failure Mode E: Network Blip . . . 49
The Resilience Toolbox . . . 49
Timeouts . . . 49
Retry Policies . . . 49
Circuit Breakers . . . 50
Fallback Strategies . . . 50
Framework Handles Semantics, Infrastructure Handles Transport . . . 51
The Durable Queue as a Resilience Primitive . . . 51

Chapter 10: Testing, Simulation, and Chaos **53**
You Cannot Set Fire to the Hotel . . . 53
Tier A: Component Tests – Test the Alarms . . . 53
Stubs . . . 53
Recording Doubles . . . 54
Contract Tests . . . 54
Tier B: Simulation – Computer-Model the Evacuation . . . 54
SimulationChatModel . . . 55
Scenario Definition . . . 55
SimulationResult . . . 56
Tier C: Chaos Engineering – Run the Drill with Real People . . . 56
Built-In, Not Bolted On . . . 56

ChaosExperiment . . . 56
Fault Types . . . 57
Assertion Types . . . 57
ChaosReport . . . 57
The Testing Pyramid . . . 58

Chapter 11: Shared State and Consistency **59**
Two Kinds of Truth . . . 59
Advisory Memory: Eventual Consistency . . . 59
Authoritative State: Choose Your Consistency . . . 60
Distributed Locking . . . 60
Optimistic Concurrency . . . 60
External Systems . . . 60
The Right Model for the Right Data . . . 61

Chapter 12: Federation – The Hotel Chain **62**
Beyond a Single Cluster . . . 62
Realms . . . 62
Capacity Advertisement . . . 63
Routing Hierarchy . . . 63
When to Federate . . . 64
Cross-Realm Authentication . . . 64
The Hotel Chain as a Design Pattern . . . 64

Chapter 13: Discovery and Evolution **66**
Finding What You Need . . . 66
Level 1: Static Configuration . . . 66
Level 2: Capability Registration . . . 66
Level 3: Dynamic Tool Catalog . . . 67
Tags . . . 67
Why Natural Language Contracts Eliminate Schema Versioning . . . 67
When Does This Break? . . . 68
Evolution Without Coordination . . . 68

Chapter 14: Infrastructure-Native Deployment **69**
Let Kubernetes Do Its Job . . . 69
Each Ensemble Is a Deployment . . . 69
Health Endpoints . . . 71
Graceful Shutdown Sequence . . . 71
The Service . . . 71
Auto-Scaling . . . 72
What the Framework Provides for K8s . . . 73
Local Development . . . 73

Chapter 15: The Road Ahead **74**
What We Have Built . . . 74
Semantic Capability Matching . . . 74
Autonomous Ensemble Spawning . . . 75

Cross-Organization Federation . . . 75
Formal Verification of Delegation Chains . . . 75
Cost-Optimal Routing . . . 76
Ensemble Learning and Adaptation . . . 76
The Bigger Picture . . . 76

Appendix A: Wire Protocol Reference **78**
Existing v2.1.0 Messages (Unchanged) . . . 78
New v3.0.0 Messages: Registration . . . 78
`ensemble_register` . . . 78
`ensemble_deregister` . . . 79
`capacity_update` . . . 79
New v3.0.0 Messages: Work Requests . . . 80
`task_request` . . . 80
`task_accepted` . . . 80
`task_progress` . . . 80
`task_response` . . . 81
`tool_request` . . . 81
`tool_response` . . . 82
New v3.0.0 Messages: Human Interaction . . . 82
`directive` . . . 82
`query` . . . 82
`query_response` . . . 82
`notification` . . . 83
New v3.0.0 Messages: Network Coordination . . . 83
`profile_applied` . . . 83
`capability_query` . . . 83
`capability_response` . . . 83

Appendix B: SPI Reference **85**
Transport Layer . . . 85
RequestQueue . . . 85
ResultStore . . . 85
Transport . . . 86
Delivery . . . 86
DeliveryHandler . . . 86
Ingress . . . 86
Result Caching . . . 87
ResultCache . . . 87
Shared State . . . 87
LockProvider . . . 87
Observability . . . 88
AuditSink . . . 88
ReviewNotifier . . . 88
Summary . . . 88

Appendix C: Comparison Matrix **90**
Multi-Agent Framework Comparison . . . 90

Microservice Pattern Comparison 92
MCP Detailed Comparison 93
Complementary Usage 94

Chapter 1: The Hotel That Never Sleeps

The Problem Nobody Is Talking About

There is a quiet assumption buried in every multi-agent AI framework built in the last three years. It goes something like this: you define agents, define tasks, wire them together, press run, and collect results. The agents live for the duration of that run. When the output arrives, everything is garbage-collected and gone.

This assumption is so deeply embedded that it is rarely questioned. The entire lifecycle – creation, execution, destruction – happens within one process, one invocation, one breath. It is the batch processing model applied to AI: compute the answer, return it, shut down.

And for many problems, it works. "Research this topic and write a report." "Analyze this dataset and produce a summary." "Draft an email based on this context." These are bounded problems with clear start and end points. A group of agents can handle them in a single execution run and produce a satisfying result.

But real-world systems are not bounded problems. They are not batch jobs. They are living, breathing operations that run continuously, handle work as it arrives, adapt to changing conditions, and involve human judgment at critical moments. And no existing multi-agent framework – not CrewAI, not AutoGen, not LangGraph, not any of the dozens of others – can model them.

Walk Into a Hotel

Consider a hotel. Not a conceptual hotel, but a real one. A 200-room property in a city center, open 24 hours a day, 365 days a year.

The hotel is composed of departments. The front desk handles check-ins, check-outs, and guest inquiries. Housekeeping cleans rooms and manages linen inventory. The kitchen prepares meals for the restaurant and room service. Room service takes orders from guests and coordinates with the kitchen for preparation and with runners for delivery. Maintenance handles repairs, preventive inspections, and emergency responses. Procurement orders supplies – everything from food to lightbulbs to replacement plumbing fixtures. Accounting reconciles revenue, manages payroll, and handles vendor payments.

Each department is autonomous. The kitchen does not need permission from the front desk to prepare a meal. Maintenance does not ask the kitchen before replacing a broken faucet. Each department has

its own staff, its own expertise, its own processes, its own tools. They are self-contained operational units.

But they are not isolated. They communicate constantly. Room service calls the kitchen: "Club sandwich, room 403." Maintenance calls procurement: "We need a replacement valve for the boiler in building 2, model X-420, urgent." The front desk broadcasts: "Guest checking out of room 201" – housekeeping hears it and schedules a cleaning, accounting hears it and closes the billing folio.

These communications happen **laterally**. Room service talks directly to the kitchen. There is no central "hotel brain" that routes every message. No one asks the general manager for permission to call the kitchen. The departments know who to call and what to ask for.

Now consider the humans. The general manager comes in at 8am. She walks around, checks on departments, reviews overnight reports. She gives some direction: "There is a VIP guest arriving in room 801 this afternoon – make sure everything is perfect." She handles a situation that needs her authority: the daily cash reconciliation requires opening the safe, and only she has the combination. At 6pm, she goes home.

The hotel does not stop.

The night manager takes over. Housekeeping continues cleaning turnover rooms. The kitchen preps for the late-night menu. Maintenance responds to a plumbing emergency in room 305. Room service handles a 2am pizza order. None of this requires the general manager's presence. She is asleep.

At 8am the next morning, she returns. Within minutes, the dashboard shows her everything that happened overnight: 47 rooms cleaned, 3 maintenance calls (one emergency), 12 room service orders, a guest complaint about noise on the 4th floor, and a pending purchase order for kitchen equipment that exceeds her pre-authorized threshold and needs her signature.

She signs the purchase order. She addresses the noise complaint. She reviews the maintenance emergency response. She gives some new direction. Then she attends a meeting, and for two hours, she is not interacting with the hotel at all. The hotel keeps running.

The Architectural Blueprint

This hotel is not a metaphor for a single concept. It is an architectural blueprint with remarkably precise mappings to distributed system design. Let us enumerate them.

A department is an ensemble. An autonomous group of specialists (agents) who handle a domain of work. The kitchen ensemble has agents for meal preparation, inventory management, menu planning. The maintenance ensemble has agents for diagnosis, repair, inspection. Each ensemble is a self-contained operational unit with its own capabilities.

Communication between departments is cross-ensemble messaging. When room service calls the kitchen, it is one ensemble sending a work request to another ensemble. The request carries context ("club sandwich, room 403"), priority (normal), and a way to get the result back ("radio me when it is ready").

The hotel directory is a service registry. Every department knows every other department exists and what it can do. When maintenance needs to order parts, it knows procurement handles that. It does not need to discover procurement dynamically for routine operations – the organizational structure is known.

A guest request is a work item. It arrives at a department (via phone, in person, or relay from another department), enters a queue, gets prioritized, gets processed, and the result is delivered.

The general manager is a human participant. She observes, directs, and makes gated decisions. She is not required for the hotel to operate. The hotel runs without her. But certain processes (opening the safe) require her specific authority.

The overnight staff change is seamless continuity. The hotel has state – guest records, room status, pending orders, maintenance logs. When the manager leaves and returns, she catches up from that state. She does not need to have been present for every event.

The hotel chain is a federation. Multiple hotels under the same brand can share resources. When Hotel A is at capacity during a conference, Hotel B across town (which is under renovation and has idle kitchen capacity) can handle overflow meal preparation requests.

What Current Frameworks Cannot Model

Let us map the hotel to existing multi-agent frameworks and see where they break.

CrewAI: You can model the kitchen as a "crew" with agents for meal prep and inventory. You run the crew, it processes tasks, it returns results, it shuts down. But you cannot have the kitchen crew running continuously, handling meal orders as they arrive throughout the day. You cannot have the room service crew call the kitchen crew to delegate a meal preparation. You cannot have the general manager drop in mid-execution to reprioritize orders. The crew is a batch job, not a service.

AutoGen: You can model the hotel as a group chat where agents exchange messages. But agents in AutoGen share a single conversation context within one process. The kitchen agents and the maintenance agents are all in the same process, the same conversation. There is no concept of independent departments running in their own processes, scaling independently, surviving restarts. And there is no way for a human to connect and disconnect from the conversation at will without the conversation depending on their presence.

LangGraph: You can model a workflow graph where nodes are agent steps and edges are transitions. The graph executes within a single process. LangGraph Cloud provides hosted execution, but the graphs are still isolated: one graph cannot delegate a complex task to another independently running graph. There is no peer-to-peer communication between deployed graphs.

MCP (Model Context Protocol): MCP provides tool-level interoperability. An agent can call a function exposed by an MCP server. But MCP is function calls, not task delegation. When maintenance calls procurement, it is not invoking a "create purchase order" function. It is delegating the entire procurement process – find vendors, compare prices, check budget, place order, track delivery – and getting back the result. MCP does not model this.

The fundamental limitation is the same across all of them: **the multi-agent ensemble is a transient, in-process construct.** It exists for the duration of one execution run. It cannot persist, cannot communicate with other ensembles, cannot handle work as a continuously running service, and cannot support human participation that is decoupled from the execution lifecycle.

Three Properties That Matter

The hotel model is valuable because it captures three properties that production AI systems need and that no current framework provides.

Property 1: Always-On

The hotel does not run in batch mode. It does not process all guest requests at once and then shut down. It runs continuously. Requests arrive at unpredictable times. Some requests take seconds (guest asks for extra towels). Some take hours (maintenance repairs a boiler). Some take days (procurement orders parts from overseas). The hotel handles all of these concurrently, continuously, indefinitely.

An AI system modeling business operations needs the same property. The "research department" does not run once and exit. It runs continuously, handling research requests as they arrive, maintaining memory of past research, building on previous findings. The "customer support department" does not process a batch of tickets and shut down. It is always on.

Property 2: Decentralized

The hotel does not have a central brain that routes every message and makes every decision. Departments communicate directly with each other. Room service talks to the kitchen. The kitchen talks to procurement. This is not chaos – it is structured autonomy. Each department knows its boundaries, knows who to call for what, and makes decisions within its domain.

Centralized orchestration is a single point of failure, a bottleneck, and an anti-pattern for systems that need to scale. When the conference generates 200 simultaneous room service orders, the kitchen needs to process them as fast as possible. Routing every order through a central coordinator adds latency and fragility.

Property 3: Human-Augmented, Not Human-Dependent

The hotel runs when no manager is present. The manager adds value when she is present – she observes, directs, and handles things that need her authority. But the system does not depend on her. It does not pause and wait for her to approve every room cleaning or every meal preparation.

Most human-in-the-loop AI systems get this wrong. They treat the human as a gatekeeper: the system pauses, presents a decision, waits for the human to approve, and only then continues. If the human is not present, the system either times out or blocks indefinitely. This makes the human a bottleneck and the system fragile.

The hotel model inverts this. The human is a **participant** in the system, not a **controller** of it. The system runs autonomously. The human joins when they choose to, observes the current state (the system catches them up instantly), gives direction, handles things that require their authority, and leaves when they choose to. The system continues.

What We Are Building

AgentEnsemble v3.0.0 – the Ensemble Network – implements this hotel model as a software architecture.

Each department becomes an ensemble: an autonomous group of AI agents deployed as a long-running Kubernetes service. Ensembles share tasks and tools with each other over a network. Cross-ensemble delegation allows one ensemble to hand off a complex, multi-step process to another ensemble and receive only the final output. Humans connect via a dashboard to observe, direct, and gate critical decisions – but the system runs without them.

The architecture is infrastructure-native: Kubernetes for deployment and scaling, durable message queues for reliable work delivery, OpenTelemetry for distributed tracing, and pluggable backends for everything. It is not a prototype or a proof of concept. It is designed for production.

The rest of this book describes the architecture in detail. Part II covers the core mechanisms: cross-ensemble delegation, the work request protocol, capacity management, and durable transport. Part III addresses human participation and observability. Part IV covers resilience and testing, including the built-in chaos engineering framework. Part V discusses advanced patterns: shared state consistency, federation, and dynamic capability discovery. Part VI covers deployment and the road ahead.

Let us start with the fundamental shift that makes all of this possible: treating ensembles as services, not scripts.

Chapter 2: Ensembles as Services

Scripts Versus Services

Every program falls into one of two categories. A **script** starts, does work, produces output, and exits. A **service** starts, runs indefinitely, handles work as it arrives, and continues until explicitly stopped.

A shell script that processes a CSV file is a script. A web server is a service. A cron job that generates a daily report is a script. A database is a service. The distinction is not about complexity – some scripts are enormously complex, and some services are simple. The distinction is about **lifecycle**: does the program own its own timeline, or does the work arriving from the outside world own it?

Every multi-agent AI framework built to date produces scripts. You define agents and tasks, call `run()`, wait for the output, and the agents cease to exist. The framework manages the lifecycle: it creates agents, executes tasks, and tears everything down.

```
// This is a script. It runs and exits.
EnsembleOutput output = Ensemble.run(model,
    Task.of("Research AI trends"),
    Task.of("Write a summary report"));
// Agents are gone. Ensemble is gone. Output is all that remains.
```

AgentEnsemble v2.x works this way. So does every other framework. It is the natural first step: get multi-agent orchestration working, then worry about deployment models.

But a hotel is not a script. The kitchen does not run once, prepare all meals that will ever be ordered, and shut down. The kitchen starts, waits for orders, prepares meals as they arrive, and runs until someone decides to close the restaurant. The kitchen is a service.

The Long-Running Ensemble

The Ensemble Network introduces a second execution mode. In addition to the existing one-shot `run()`, an ensemble can now `start()`:

```
Ensemble kitchen = Ensemble.builder()
    .name("kitchen")
    .chatLanguageModel(model)
    .task(Task.of("Manage kitchen operations"))
    .shareTask("prepare-meal", Task.builder()
```

```
        .description("Prepare a meal as specified")
        .expectedOutput("Confirmation with preparation details and timing")
        .build())
    .shareTool("check-inventory", inventoryTool)
    .build();

kitchen.start(7329);
```

When `start()` is called, the ensemble:

1. Starts a WebSocket server on the specified port
2. Transitions to the READY lifecycle state
3. Registers its shared capabilities (tasks and tools) so other ensembles can discover them
4. Begins accepting incoming work requests
5. Processes work through a priority queue
6. Runs indefinitely until explicitly stopped

The one-shot `run()` method is unchanged. Existing code continues to work exactly as before. The long-running mode is additive – a new capability, not a replacement.

What an Ensemble Shares

A long-running ensemble declares what it offers to the network. There are two sharing primitives.

Shared Tasks

A shared task is a named, complete process that other ensembles can trigger. The target ensemble runs the full task with its own agents, tools, memory, and review gates. The caller hands off the work and gets back a result.

```
.shareTask("prepare-meal", Task.builder()
    .description("Prepare a meal as specified by the order")
    .expectedOutput("Confirmation with preparation details, timing, and ticket number")
    .tools(ovenTool, grillerTool, platePresentationTool)
    .build())
```

When another ensemble triggers "prepare-meal," the kitchen ensemble synthesizes an agent for the task (or uses an explicitly configured agent), runs the full execution pipeline – including the ReAct loop, tool calls, structured output parsing, and any configured review gates – and returns the output.

This is fundamentally different from calling a function. A function call is atomic: call, compute, return. A shared task is a delegation: "handle this for me, using whatever process you use, and tell me the result." The caller does not know or care about the internal process. It is like asking the kitchen to make a sandwich versus operating a sandwich-making machine yourself.

Shared Tools

A shared tool is a specific capability that other ensembles' agents can use directly in their reasoning loop. The tool executes on the owning ensemble's side but returns results to the calling agent.

```
.shareTool("check-inventory", inventoryTool)
.shareTool("dietary-check", allergyCheckTool)
```

Shared tools are lighter weight than shared tasks. The calling agent maintains control of its own reasoning process but borrows a specific capability from another ensemble. It is the difference between hiring a department (shared task) and borrowing a tool (shared tool).

An agent in the room service ensemble might use `check-inventory` mid-reasoning to verify that wagyu beef is in stock before placing a kitchen order. The agent decides what to do with the result. It stays in its own ReAct loop.

Consuming Shared Capabilities

Other ensembles consume shared capabilities using `NetworkTask` and `NetworkTool`, both of which implement the existing `AgentTool` interface. This means agents do not know whether a tool is local or remote. The abstraction is seamless.

```
Ensemble roomService = Ensemble.builder()
    .name("room-service")
    .chatLanguageModel(model)
    .task(Task.builder()
        .description("Handle guest room service request")
        .tools(
            NetworkTask.from("kitchen", "prepare-meal"),
            NetworkTool.from("kitchen", "check-inventory"),
            NetworkTool.from("kitchen", "dietary-check"),
            NetworkTask.from("maintenance", "repair-request"))
        .build())
    .build();

roomService.start(7329);
```

When the room service agent's ReAct loop decides to call `prepare-meal`, the agent does not know that it is sending a work request over a WebSocket connection to a kitchen pod in a different Kubernetes deployment. It calls a tool, gets a result, and continues reasoning. The framework handles serialization, transport, waiting, timeout, and deserialization.

Three Task Modes

A long-running ensemble has three categories of tasks.

Shared (Reactive)

Shared tasks are triggered by external work requests. They execute when another ensemble (or a human, or an external system) asks for them.

"Kitchen, prepare a club sandwich" is reactive. Nobody asked the kitchen to prepare a sandwich proactively – it was triggered by a guest order relayed through room service.

Scheduled (Proactive)

Scheduled tasks run on a timer, producing output that is broadcast to anyone listening.

```
.scheduledTask(ScheduledTask.builder()
    .name("inventory-report")
    .task(Task.of("Check current inventory levels and report any shortages"))
    .schedule(Schedule.every(Duration.ofHours(1)))
    .broadcastTo("hotel.inventory")
    .build())
```

This is the ticket seller on the street. Nobody asked for the inventory report. The kitchen produces it every hour and broadcasts it to the `hotel.inventory` topic. Procurement subscribes to that topic and pre-orders supplies when levels are low. The front desk subscribes to know whether the restaurant can offer the full menu.

Scheduled tasks introduce a **push** model alongside the existing **pull** model. Ensembles do not just respond to requests – they proactively produce and distribute information.

Internal (Private)

Internal tasks are part of the ensemble's own operational workflow. They are not shared with the network and are not visible to other ensembles. The kitchen's internal task for "plan tomorrow's specials based on inventory" is private. It runs inside the kitchen ensemble and its output stays inside the kitchen ensemble.

The Network

When multiple ensembles start on a network, they form a mesh. Each ensemble is a node. The connections between them are the shared task/tool relationships.

```
+-------------+    prepare-meal    +-------------+
| room-svc    |------------------>| kitchen      |
|             |<--check-inventory--|             |
+-------------+                    +-------------+
      |                                   |
      | repair-request                    | order-supplies
      v                                   v
+-------------+                    +-------------+
| maintenance |------------------>| procurement  |
|             |   purchase-parts   |             |
+-------------+                    +-------------+
```

This is not a centrally orchestrated pipeline. There is no "hotel main" program that defines the execution order. Each ensemble runs independently. The connections emerge from the shared capabilities each ensemble declares and the tools each ensemble consumes.

Room service calls kitchen when it needs to. Kitchen calls procurement when it needs to. Maintenance calls procurement when it needs to. These are lateral, peer-to-peer communications. No central coordinator.

What Does Not Change

The one-shot execution model (`Ensemble.run()`) is entirely unchanged. Existing code, existing tests, existing examples all work exactly as before. The v3.0.0 changes are purely additive.

The core abstractions – `Task`, `Agent`, `AgentTool`, `EnsembleOutput`, `ReviewHandler`, `MemoryStore`, `ExecutionTrace` – are unchanged. `NetworkTask` and `NetworkTool` implement `AgentTool`. The existing ReAct loop, agent synthesis, structured output parsing, tool execution, metrics, and tracing all work the same way.

The v2.x ensemble is a perfectly good building block. v3.0 wraps it in a service layer that adds network communication, lifecycle management, and shared capabilities. The inner execution engine is the same.

From Script to Service: The Mental Model Shift

The hardest part of adopting this architecture is not the code. It is the mental model.

In the script model, you think in terms of workflows: "First, research. Then, write. Then, edit." You define the pipeline, run it, get the output. You are the orchestrator.

In the service model, you think in terms of capabilities: "The kitchen can prepare meals. Room service can handle guest orders. Maintenance can fix things." You define what each department does, start them all, and let work flow through the network as it arrives. You are not the orchestrator – the work requests are.

This is the same shift that happened in software architecture when web applications moved from monolithic request handlers to microservices. The monolith processes the entire request. The microservice processes one domain concern and calls other services for the rest. The shift is from "I control the flow" to "I declare my capabilities and respond to requests."

For AI agent systems, this shift is overdue.

Chapter 3: Cross-Ensemble Delegation

Borrowing a Tool Versus Hiring a Department

When you need a hole drilled in a wall, you have two options. You can borrow a drill from your neighbor, drill the hole yourself, and return the drill. Or you can hire a contractor, tell them where you want the hole, and let them handle it. You get the same result – a hole in the wall – but the process is fundamentally different.

Borrowing the drill is a **tool call**. You maintain control of the process. You decide the angle, the depth, the bit size. The drill is a capability you temporarily acquire and operate yourself.

Hiring the contractor is a **task delegation**. You hand off the entire process. The contractor brings their own drill, their own experience, their own judgment about the right approach. They might discover the wall is concrete and switch to a masonry bit. They might notice a pipe behind the wall and adjust the position. You do not micromanage. You get the hole.

This distinction maps precisely to the two sharing primitives in the Ensemble Network.

NetworkTool is borrowing the drill. An agent in one ensemble calls a specific tool hosted by another ensemble. The calling agent stays in control of its reasoning loop. It uses the tool result to inform its next step.

NetworkTask is hiring the contractor. An agent in one ensemble delegates an entire task to another ensemble. The target ensemble runs its full process – agent synthesis, ReAct loop, tool calls, memory access, review gates, potentially sub-delegations to other ensembles – and returns the output. The calling agent receives only the final result.

Most existing cross-service AI communication is tool-level. MCP provides tool calls between services. LangChain tool integrations are function calls. The Ensemble Network provides both tool calls and task delegation. The task delegation is what makes it fundamentally different.

What Happens During a Cross-Ensemble Task Delegation

Let us trace the exact sequence of events when a maintenance ensemble delegates parts ordering to a procurement ensemble.

Step 1: The maintenance agent is working on a task: "Fix the boiler in building 2." It has diagnosed the problem – a faulty valve – and needs a replacement part.

Step 2: The agent's LLM decides to call the `purchase-parts` tool. From the agent's perspective, this is just another tool in its ReAct loop, no different from calling a local calculator or web search

tool.

```
Thought: I need to order a replacement valve. I'll use the purchase-parts tool.
Action: purchase-parts
Action Input: "Order replacement valve model X-420 for building 2 boiler, urgent priority"
```

Step 3: The `NetworkTask` implementation serializes this into a WorkRequest:

```
{
  "type": "task_request",
  "requestId": "maint-7721-valve-order",
  "from": "maintenance",
  "task": "purchase-parts",
  "context": "Order replacement valve model X-420 for building 2 boiler, urgent priority",
  "priority": "HIGH",
  "deadline": "PT30M",
  "delivery": { "method": "WEBSOCKET", "address": "ws://maintenance:7329/ws" },
  "traceContext": {
    "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01"
  }
}
```

Step 4: The WorkRequest is sent to the procurement ensemble (via WebSocket or durable queue). The maintenance agent's thread blocks, waiting for the response (in AWAIT mode).

Step 5: The procurement ensemble receives the request. It looks up the "purchase-parts" shared task definition.

Step 6: Procurement synthesizes an agent for the task (using the configured `AgentSynthesizer`, which might produce a "Procurement Specialist" persona). The agent receives the context: "Order replacement valve model X-420 for building 2 boiler, urgent priority."

Step 7: The procurement agent executes its own ReAct loop. It might: - Call a vendor search tool to find suppliers for model X-420 - Call a price comparison tool to compare quotes - Check the budget allocation tool to verify funds are available - Call a purchase order tool to place the order - Maybe even delegate shipping logistics to a logistics ensemble (another cross-ensemble delegation, creating a chain)

Step 8: The procurement agent completes. The output is: "Ordered from SupplyCo, PO #4821, estimated delivery Thursday. Total cost: $342.00, charged to maintenance budget code M-2026-B2."

Step 9: The procurement ensemble sends the response back to maintenance:

```
{
  "type": "task_response",
  "requestId": "maint-7721-valve-order",
  "status": "COMPLETED",
  "result": "Ordered from SupplyCo, PO #4821, estimated delivery Thursday. Total cost: $342.00
}
```

Step 10: The maintenance agent's `NetworkTask` tool call unblocks. The agent receives the result and continues its ReAct loop:

```
Observation: Ordered from SupplyCo, PO #4821, estimated delivery Thursday.
Thought: The replacement valve is ordered and arriving Thursday. I should schedule the
repair for Thursday afternoon and notify the building manager.
Action: ...
```

From the maintenance agent's perspective, it called a tool and got a result. It does not know that behind that tool call, an entire procurement ensemble ran a multi-step process involving vendor search, price comparison, budget verification, and order placement. This is the power of the abstraction.

The Delegation Chain

Cross-ensemble delegation is naturally recursive. In step 7 above, the procurement agent might delegate shipping to a logistics ensemble. The logistics ensemble might delegate customs clearance to a compliance ensemble. Each ensemble in the chain runs its own complete process.

```
maintenance
  -> procurement (purchase-parts)
       -> logistics (arrange-shipping)
            -> compliance (customs-clearance)
            <- clearance approved
       <- shipping arranged, tracking #SH-8891
  <- ordered, PO #4821, delivery Thursday
```

The maintenance ensemble sees only the final result. It does not know about the delegation chain underneath. Each ensemble in the chain sees only its direct caller and its direct delegates.

This mirrors how organizations work. The maintenance manager asks procurement to order parts. She does not need to know that procurement used a specific vendor, that the vendor ships from overseas, or that customs clearance was required. She needs to know: will the part arrive, and when?

Depth Limiting

Delegation chains need bounds. The existing `maxDelegationDepth` configuration (used for agent-to-agent delegation within a single ensemble) extends to cross-ensemble delegation. The depth counter propagates with the trace context. If the chain exceeds the configured maximum, the framework returns an error rather than allowing infinite recursion.

NetworkTool: The Lighter Alternative

Not every cross-ensemble interaction needs full task delegation. Sometimes you just need to check something.

The room service agent is assembling an order for a guest. Before calling the kitchen, it wants to verify that the requested dish is available:

```
Thought: Guest wants wagyu steak. Let me check if kitchen has it in stock.
Action: check-inventory
Action Input: "wagyu beef"
```

The `NetworkTool` serializes this as a tool request (not a task request):

```
{
  "type": "tool_request",
  "requestId": "rs-8801",
  "from": "room-service",
  "tool": "check-inventory",
  "input": "wagyu beef"
}
```

The kitchen ensemble receives this, executes the `inventoryTool` directly (no agent synthesis, no ReAct loop, no task pipeline), and returns:

```
{
  "type": "tool_response",
  "requestId": "rs-8801",
  "result": "Yes, 3 portions available"
}
```

The room service agent continues:

```
Observation: Yes, 3 portions available
Thought: Wagyu is in stock. I can proceed with the order.
Action: prepare-meal
Action Input: "Wagyu steak, medium-rare, room 403"
```

Notice the pattern: the agent used a `NetworkTool` (check-inventory) for a quick lookup, then a `NetworkTask` (prepare-meal) for the actual delegation. Both are tools in its ReAct loop. The agent chose which to use based on what it needed: a fact check versus a full process delegation.

Transparent to the Agent

The most important design property of cross-ensemble delegation is that it is transparent to the agent.

Both `NetworkTask` and `NetworkTool` implement the existing `AgentTool` interface. They have names, descriptions, and execute methods. The LLM sees them as tools in its available tool list. It decides when to call them based on the descriptions, just like it decides when to call a local calculator tool or a web search tool.

```
// These three tools are indistinguishable to the agent:
Task.builder()
    .description("Handle guest request")
    .tools(
        calculatorTool,                                    // local tool
        NetworkTool.from("kitchen", "check-inventory"),    // remote tool
        NetworkTask.from("kitchen", "prepare-meal"))       // remote task delegation
    .build();
```

The agent does not know which tools are local, which are remote tool calls, and which are full task delegations. It does not need to know. The LLM reasons about what it needs to do, selects the

appropriate tool, calls it, and uses the result.

This transparency means that the entire existing infrastructure – the ReAct loop, tool executor, metrics, tracing, error handling, guardrails – works unchanged with network tools. A `NetworkTask` call appears in the `ExecutionTrace` the same way a local tool call does. Metrics record its duration. The trace captures its input and output. Guardrails can validate its result.

When to Use Which

Situation	Use	Hotel analogy
Need a quick fact	`NetworkTool`	"Is the restaurant open?"
Need a specific calculation or lookup	`NetworkTool`	"What's the room rate for tonight?"
Need a complete process handled	`NetworkTask`	"Prepare a meal for room 403"
Need a complex, multi-step workflow	`NetworkTask`	"Order spare parts for the boiler"
Need to maintain control of reasoning	`NetworkTool`	Borrow the drill
Need to hand off responsibility	`NetworkTask`	Hire the contractor

The choice is the agent's. The LLM, guided by the tool descriptions, decides whether it needs a fact (tool) or a delegation (task). The framework provides both options; the agent selects based on its reasoning.

Chapter 4: The WorkRequest Protocol

A Work Order, Not a Function Call

In a traditional microservice architecture, services communicate via API calls: HTTP requests with typed JSON payloads, versioned endpoints, and schema contracts. When the schema changes, clients break. This is the tyranny of typed contracts in distributed systems.

In a hotel, work orders do not look like API calls. When room service tells the kitchen "Club sandwich, room 403, no tomato, add extra bacon," nobody checks a JSON schema. The kitchen understands the request because the language is clear and the context is shared. If room service says "the usual for the guest in 403" and the kitchen knows what that means from past experience, that works too.

The Ensemble Network's communication primitive is the **WorkRequest**: a standardized envelope that carries work between ensembles. It is not a function call. It is a work order. The content is natural language. The envelope provides the operational metadata: who is asking, what priority, when it is needed, and how to deliver the result.

The Envelope

```
public record WorkRequest(
    String requestId,
    String from,
    String task,
    String context,
    Priority priority,
    Duration deadline,
    DeliverySpec delivery,
    String traceContext,
    CachePolicy cachePolicy,
    String cacheKey
) {}
```

Each field serves a specific purpose:

requestId: The unique identifier for this specific work request. Generated by the caller. Serves two roles: correlation (matching the response to the request) and idempotency (if the same request is sent twice due to a retry, the receiver recognizes the duplicate and returns the cached result rather than executing again).

from: The name of the requesting ensemble. Used for routing the response and for audit trail logging.

task: The name of the shared task or tool to execute. Must match a name declared via `shareTask()` or `shareTool()` on the target ensemble.

context: The natural language input for the task. This is the "what to do" – the content of the work order. For a NetworkTask, this is the full task context that the target ensemble's agent will work with. For a NetworkTool, this is the tool input.

priority: CRITICAL, HIGH, NORMAL, or LOW. Determines ordering in the target ensemble's priority queue. CRITICAL work is processed before HIGH, which is processed before NORMAL, and so on.

deadline: The caller's SLA. "I need this within 30 minutes." The target ensemble is not required to meet the deadline – it reports its estimated completion time, and the caller decides whether to wait, cancel, or try elsewhere.

delivery: How and where to return the result. A `DeliverySpec` containing a method (WebSocket, queue, topic, webhook, store, broadcast-claim, or none) and a method-specific address. This decouples the response path from the request path.

traceContext: W3C Trace Context headers (`traceparent` and `tracestate`) for distributed tracing. Mandatory. Allows the cross-ensemble call to be correlated in Jaeger, Tempo, or any OpenTelemetry-compatible backend.

cachePolicy: USE_CACHED or FORCE_FRESH. When USE_CACHED, the target ensemble may return a previously computed result for the same `cacheKey` if it is within the `maxAge`. When FORCE_FRESH, the target always executes.

cacheKey: Optional key for result caching. Different from `requestId` (which is for idempotency). The cache key represents "this type of request" rather than "this specific request instance."

"Call Me Back at This Number"

The delivery specification is the most novel part of the WorkRequest. In traditional request-response systems, the response goes back the way the request came in. An HTTP response goes back on the same TCP connection. A WebSocket response goes back on the same WebSocket.

In the Ensemble Network, the response path is explicitly specified by the caller. This enables a pattern familiar from everyday life: "Call me back at this number when you have a hotel room available."

The caller says: "Here is my work request. When you have the result, deliver it to `kafka://maintenance.results`." The target ensemble processes the request, and when the result is ready, publishes it to the specified Kafka topic. The caller (or any replica of the caller) consumes it from there.

This decoupling is essential for production deployment. In Kubernetes, pods come and go. The pod that sent the request might be killed before the response arrives. The pod that processes the request might be a different replica than the one that accepted it. The response needs to reach the caller regardless of which pods are currently running. The delivery specification makes this explicit.

Delivery Methods

WEBSOCKET: Direct, real-time delivery over a WebSocket connection. The simplest method. Used in development and for fast, synchronous interactions where both parties are connected.

QUEUE: Point-to-point delivery via a durable queue (Redis Streams, SQS). The result is enqueued; exactly one consumer processes it. Survives pod restarts.

TOPIC: Pub/sub delivery via a message topic (Kafka). The result is published; all subscribers receive a copy. Useful when multiple ensembles need the same result, or when the audit system needs a copy.

WEBHOOK: HTTP POST to a specified URL. Useful for integrating with external systems that are not part of the ensemble network.

STORE: Write the result to a shared key-value store (Redis) keyed by the `requestId`. The caller polls or subscribes for the result. Useful for very large results or when the caller wants to retrieve the result at its convenience.

BROADCAST_CLAIM: Announce the result to all replicas of the requesting service. The first replica to claim it receives the full payload. Useful when the caller has multiple replicas and the result should be processed by whichever one is free.

NONE: Fire and forget. The caller does not need a response. Useful for notifications or background processes.

Ingress: How Work Arrives

The delivery specification controls how results leave an ensemble. The ingress configuration controls how work arrives.

A long-running ensemble can accept work from multiple sources simultaneously:

WebSocket: Direct from another ensemble over a persistent WebSocket connection. The simplest and fastest ingress. Used when ensembles have direct connections.

Queue: Pull from a durable queue. The ensemble polls or subscribes to a queue (Redis Streams, Kafka, SQS) and picks up work items. Multiple replicas of the same ensemble can read from the same queue with consumer groups, providing natural load balancing.

HTTP API: Accept work via `POST /api/work`. Useful for external systems, scripts, CI pipelines, or any client that can make HTTP requests. The request body is a WorkRequest JSON.

Topic subscription: React to events published on a topic. When the front desk publishes a "guest checkout" event to `hotel.events`, the housekeeping ensemble (subscribed to that topic) receives it and triggers a room cleaning task.

Schedule: Internal cron or interval. Scheduled tasks generate WorkRequests internally on a timer, without any external trigger.

All ingress sources normalize their input to the same `WorkRequest` envelope. Once normalized, the work item enters the ensemble's priority queue and is processed in priority order regardless of how it arrived.

The Request-Response Cycle

Putting it all together, a typical cross-ensemble interaction follows this cycle:

```
1. Caller constructs WorkRequest
   (requestId, task, context, priority, deadline, delivery, trace)

2. WorkRequest is sent via the chosen ingress method
   (WebSocket direct, queue, HTTP, topic)

3. Target ensemble receives and acknowledges
   -> task_accepted { requestId, queuePosition, estimatedCompletion }

4. Target ensemble processes the request
   (agent synthesis, ReAct loop, tool calls, review gates)

5. Target ensemble delivers the result
   via the delivery method specified in the WorkRequest

6. Caller receives the result
   (unblocks if AWAIT mode, triggers callback if ASYNC mode)
```

The acknowledgment in step 3 is important. The target ensemble does not just silently accept the work. It responds immediately with its queue position and estimated completion time. This gives the caller the information it needs to make routing decisions: wait, cancel and try elsewhere, or continue without the result.

Why Natural Language Context?

The `context` field in a WorkRequest is natural language, not a typed schema. This is a deliberate design decision that eliminates an entire category of distributed systems problems.

In a traditional microservice, changing a field name in a request payload breaks all callers. Adding a required field breaks all callers. Removing a field breaks all callers. Schema versioning, backward compatibility, and API evolution consume enormous engineering effort.

In the Ensemble Network, the context is: "Order replacement valve model X-420 for building 2 boiler, urgent priority." If the procurement ensemble changes how it parses this – maybe it now extracts structured data more intelligently, or handles a wider variety of request formats – the caller is unaffected. The caller sent natural language. The receiver interpreted it.

The LLM on the receiving end is the **compatibility layer**. It interprets the natural language context regardless of exact wording, phrasing, or format. Minor changes do not break callers. This is not a weakness of the protocol – it is a strength. It trades the precision of typed schemas for the resilience of natural language interpretation, which is the right trade for LLM-based systems.

When structured data is needed for specific fields (like a purchase order number or a delivery date), the structured output mechanism (`outputType`) handles it. But the task description and the context – the "what to do" – remains natural language.

Chapter 5: Capacity Management – Bend, Don't Break

The Instinct to Reject

When a web server is overwhelmed, the standard response is to reject requests. Return HTTP 503 Service Unavailable. Drop the connection. Shed load. The reasoning is sound for real-time systems: if you cannot serve the request within the expected latency window, returning an error is better than making the client wait indefinitely.

This instinct is wrong for AI agent systems.

An LLM task is not a web request. A web request expects a response in milliseconds. An LLM task might take 30 seconds, 5 minutes, or an hour. The caller already expects latency. When maintenance delegates parts ordering to procurement, it does not expect an instant response. It expects procurement to do its job and deliver the result when it is ready.

Rejecting a work request because the ensemble is busy is like a hotel telling a guest "We cannot take your room service order because the kitchen is currently preparing other meals." No hotel does this. The kitchen takes the order, puts it in the queue, and prepares it when capacity is available. The guest is told: "Your order will be ready in approximately 30 minutes." The guest decides whether to wait.

This is the "bend, don't break" principle: **accept and queue by default, reject only at hard limits.**

Accept, Queue, Estimate, Process, Deliver

When a work request arrives at an ensemble, the processing pipeline is:

```
Request arrives
  -> Is the ensemble alive and in READY state?
     No  -> Return UNAVAILABLE (the only legitimate rejection)
     Yes -> Accept into priority queue
            -> Return task_accepted with queue position and ETA
            -> Process when capacity is available
            -> Deliver result via caller-specified delivery method
```

The ensemble always accepts the work (as long as it is alive). It always reports its current queue position and estimated completion time. The caller receives this information immediately and makes

its own decision.

This is fundamentally different from the traditional approach where the server decides to accept or reject. In the Ensemble Network, the server always accepts. The client decides whether the wait is acceptable.

Caller-Side SLAs

Every WorkRequest carries a `deadline` field: the caller's SLA. "I need this within 30 minutes." This is not an instruction to the provider – it is information for the caller's own routing logic.

```
{
  "requestId": "maint-7721",
  "task": "purchase-parts",
  "deadline": "PT30M",
  "priority": "HIGH"
}
```

The provider responds with its honest estimate:

```
{
  "type": "task_accepted",
  "requestId": "maint-7721",
  "queuePosition": 7,
  "estimatedCompletion": "PT45M"
}
```

The estimated completion time (45 minutes) exceeds the caller's deadline (30 minutes). The caller's `NetworkTask` implementation now has three options:

1. **Wait anyway**: The deadline was advisory. The caller accepts the longer wait.
2. **Cancel and reroute**: Try a different provider. In a federated network, another realm's procurement ensemble might have shorter queues.
3. **Continue without**: The deadline was a best-effort target. The caller proceeds with partial information and incorporates the result later if it arrives.

The choice depends on the request mode configured by the caller (AWAIT, ASYNC, or AWAIT_WITH_DEADLINE) and the application logic. The framework provides the information; the application makes the decision.

Priority Queuing

Not all work is equal. When the hotel has 20 room service orders queued and the general manager requests a meal for a VIP guest in the penthouse, that order goes to the front.

The ensemble's internal queue orders work by priority:

`CRITICAL > HIGH > NORMAL > LOW`

Within the same priority level, work is processed in FIFO order (first in, first out).

Priority Aging

A LOW priority request submitted at 9am should not still be waiting at 5pm because higher priority requests keep arriving. Priority aging ensures that old low-priority work eventually gets processed.

The aging algorithm is configurable. The simplest approach: every N minutes, unprocessed work gets promoted one level. A LOW request submitted at 9am becomes NORMAL at 9:30am, HIGH at 10am, and CRITICAL at 10:30am. The exact intervals are configurable per ensemble.

Human Re-Prioritization

A human connected to the dashboard can see the queue and manually re-prioritize individual work items. "Move request #4821 to the front" – the request gets promoted to CRITICAL and jumps ahead of everything else.

This is the hotel equivalent of the manager walking into the kitchen and saying "This order first." The manager does not take over the kitchen. She gives a directive and walks away. The kitchen processes it.

Operational Profiles

Hotels anticipate load changes. A 500-person conference arriving on Friday means the kitchen needs extra staff, room service needs more runners, and the front desk needs all hands on deck. The hotel does not wait for the conference to arrive and then scramble. It plans ahead.

Operational profiles are pre-configured capacity plans for anticipated scenarios:

```
NetworkProfile sportingEvent = NetworkProfile.builder()
    .name("sporting-event-weekend")
    .ensemble("front-desk", Capacity.replicas(4).maxConcurrent(50))
    .ensemble("kitchen", Capacity.replicas(3).maxConcurrent(100))
    .ensemble("room-service", Capacity.replicas(3).maxConcurrent(80))
    .ensemble("maintenance", Capacity.replicas(1).maxConcurrent(10))
    .preload("kitchen", "inventory", "Extra beer and ice stocked")
    .build();

NetworkProfile offPeak = NetworkProfile.builder()
    .name("weekday-off-peak")
    .ensemble("front-desk", Capacity.replicas(1).maxConcurrent(10))
    .ensemble("kitchen", Capacity.replicas(1).maxConcurrent(20))
    .ensemble("maintenance", Capacity.dormant())
    .build();
```

A profile specifies: - **Replica counts**: How many pods of each ensemble should be running - **Concurrent task limits**: Maximum simultaneous task executions per ensemble - **Dormant flag**: Whether an ensemble should be shut down entirely (saving resources) - **Pre-loading**: Context to inject into shared memory before the scenario begins

Profiles can be applied: - **Manually**: A human sends a directive via the dashboard - **On a schedule**: "Apply `sporting-event-weekend` at Friday 4pm" - **Via rules**: "When front-desk queue depth exceeds 20, apply `high-load`"

When a profile is applied, the framework adjusts Kubernetes HPA targets, activates dormant ensembles, and pre-loads shared memory. Ensembles that need to scale up do so via K8s; the framework provides the configuration.

Elastic Scaling Within a Profile

Within a profile's configured bounds, ensembles scale dynamically based on actual load. The framework exposes Micrometer/Prometheus metrics that Kubernetes HPA uses for scaling decisions:

```
agentensemble_active_tasks{ensemble="kitchen"} 8
agentensemble_queued_requests{ensemble="kitchen"} 15
agentensemble_capacity_utilization{ensemble="kitchen"} 0.85
```

HPA watches these metrics and scales replicas within the profile's min/max bounds. If the profile says kitchen should have 1-3 replicas and the queue depth exceeds the threshold, HPA scales from 1 to 2, then to 3. If load drops, it scales back down.

The framework exposes the metrics. K8s handles the scaling. There is no custom auto-scaler in the framework – just honest reporting of the ensemble's state.

Hard Limits

The hotel has 200 rooms. No amount of scaling changes this. When all rooms are booked, the answer is "we are full."

Every ensemble can declare a hard limit: the maximum queue depth beyond which new work is rejected. This is the only scenario where the ensemble returns a rejection rather than an acceptance.

```json
{
  "type": "task_response",
  "requestId": "maint-7722",
  "status": "REJECTED",
  "reason": "AT_CAPACITY",
  "currentQueue": 200,
  "suggestion": "Try kitchen-west-wing in realm hotel-airport"
}
```

The rejection includes the current queue depth and, when available in a federated network, a suggestion of an alternative provider. The caller can use this information to reroute.

Hard limits should be generous. The "bend, don't break" principle means most load spikes are absorbed by the queue. Hard limits are a safety valve for truly exceptional situations, not a normal operating mode.

The Ticket Seller on the Street

There is a special case of capacity management that is not about handling incoming requests but about proactively distributing information.

A ticket seller standing on a street corner shouting "Tickets available!" is not responding to requests. Nobody asked. The seller is broadcasting availability. Interested parties hear the announcement and come forward if they want a ticket.

This maps to scheduled tasks with broadcast delivery. The front desk ensemble runs a scheduled task every 5 minutes: "Check current room availability." The result is broadcast to the `hotel.availability` topic. The sales system subscribes and updates the booking website. The conference coordinator subscribes and adjusts room block allocations. The manager's dashboard subscribes and shows the current count.

Nobody requested this information. The front desk produced it proactively and made it available to anyone who cares. This is the push model complementing the pull model. Some information is more valuable when it is pushed than when it must be requested.

Chapter 6: Durable Transport

The Fragility of Direct Connections

WebSocket is a beautiful protocol. Bidirectional, real-time, low-overhead. The existing AgentEnsemble v2.1.0 live dashboard is built on it: events stream from the JVM to the browser, review decisions flow from the browser to the JVM, all over a single persistent connection.

But WebSocket has a fundamental limitation: it is ephemeral. The connection exists between two specific endpoints – this pod and that browser tab, this pod and that other pod. When either endpoint restarts, the connection is gone. When a Kubernetes deployment rolls out new pods, all connections to the old pods are severed. When a network blip interrupts connectivity for five seconds, the connection drops.

For the live dashboard, this is manageable. The browser reconnects with exponential backoff, receives a late-join snapshot, and rebuilds its state. No work is lost because the browser is a viewer, not a worker.

For cross-ensemble work delivery, it is not manageable. If maintenance sends a "purchase parts" request to procurement over a WebSocket connection, and procurement's pod restarts before the request is processed, the request is lost. Maintenance's agent is blocked on a `CompletableFuture` that will never resolve. The work vanishes.

This is why the Ensemble Network separates the transport into two layers: **real-time** (WebSocket) for events and human interaction, and **durable** (message queues and result stores) for reliable work delivery.

Two Layers, One Protocol

The WorkRequest envelope is the same regardless of transport layer. A `task_request` message looks identical whether it is sent over WebSocket or enqueued in a Kafka topic. The separation is about reliability guarantees, not protocol differences.

Real-Time Layer (WebSocket)

Used for: - Streaming task progress events to the dashboard - Human directives and queries - Review gate requests and decisions - Capability handshake between ensembles - Low-latency tool calls when both parties are connected - Development and local testing

Properties: - Bidirectional, low latency - Ephemeral: does not survive pod restarts - Best-effort delivery: if the connection is down, the message is lost

Durable Layer (Message Queues + Result Store)

Used for: - Work request delivery (task_request, tool_request) - Work result delivery (task_response, tool_response) - Idempotency cache - Result caching

Properties: - Survives pod restarts, network blips, and rolling deployments - At-least-once delivery (with idempotency for exactly-once semantics) - Consumer groups for load balancing across replicas - Persistent: messages are retained until consumed (or TTL expires)

The Asymmetric Routing Problem

When work delivery uses durable queues, a subtle problem emerges: the pod that processes a request may be different from the pod that received it.

Consider procurement with 3 replicas. A work request arrives in the queue `procurement.inbox`. Replica 2 picks it up (consumer group load balancing). Replica 2 processes the request – its agent finds vendors, compares prices, places the order. Now replica 2 needs to deliver the result to maintenance.

But replica 2 does not have a WebSocket connection to maintenance. Replica 1 might have one, or maybe none of the replicas do. The request arrived via queue; the response needs to go somewhere.

This is why the WorkRequest carries a `DeliverySpec`. The caller explicitly says: "Deliver the result to `redis://maintenance.results/maint-7721`." Replica 2 writes the result to the shared result store (Redis) keyed by the request ID. Maintenance reads it from there.

The request path and the response path are completely decoupled. They may use different transport mechanisms, different infrastructure, and different pods. The `requestId` is the correlation key that ties them together.

```
Request path:
  Maintenance Pod 1 -> [procurement.inbox queue] -> Procurement Pod 2

Response path:
  Procurement Pod 2 -> [Redis result store: key=maint-7721] -> Maintenance Pod 3
```

Notice that Maintenance Pod 3 (not Pod 1) reads the result. Pod 1 might have been killed during a deployment rollout. The result is in the shared store; any maintenance replica can retrieve it.

The Transport SPI

The transport layer is pluggable. Two implementations are provided:

Simple Transport (Development)

In-process queues and direct WebSocket. No external infrastructure required. Work requests are held in a `ConcurrentLinkedQueue` in memory. Results are delivered directly over WebSocket connections.

```
EnsembleNetwork.builder()
    .transport(Transport.websocket())
    .build();
```

This is the default for local development. It is fast, simple, and requires no setup. But it does not survive process restarts, and it does not support horizontal scaling (each JVM has its own queue).

Durable Transport (Production)

External request queue and result store. Work requests are enqueued in a durable message system; results are written to a shared key-value store.

```
EnsembleNetwork.builder()
    .transport(Transport.durable(
        RequestQueue.redis(redisClient),
        ResultStore.redis(redisClient)))
    .build();
```

Or with Kafka:

```
EnsembleNetwork.builder()
    .transport(Transport.durable(
        RequestQueue.kafka(kafkaProperties),
        ResultStore.redis(redisClient)))
    .build();
```

The SPI separates the queue (how work is delivered) from the result store (where results are written). You can mix implementations: Kafka for the queue, Redis for the result store.

Custom Transport

The SPI is open. If your organization uses a different message system (RabbitMQ, AWS SQS, Google Pub/Sub), you implement the `RequestQueue` and `ResultStore` interfaces:

```
public interface RequestQueue {
    void enqueue(String queueName, WorkRequest request);
    WorkRequest dequeue(String queueName, Duration timeout);
    void acknowledge(String queueName, String requestId);
}

public interface ResultStore {
    void store(String requestId, WorkResponse response, Duration ttl);
    WorkResponse retrieve(String requestId);
    void subscribe(String requestId, Consumer<WorkResponse> callback);
}
```

The interfaces are intentionally minimal. They abstract the essential operations without leaking implementation details.

Consumer Groups and Horizontal Scaling

When an ensemble has multiple replicas, all replicas read from the same request queue using consumer groups. This is a standard pattern in both Redis Streams and Kafka:

- The queue has one logical name: `kitchen.inbox`

- Multiple consumers (kitchen pods) read from it
- Each message is delivered to exactly one consumer
- If a consumer dies without acknowledging, the message is redelivered to another consumer

This provides natural load balancing: work is distributed across replicas without any custom routing logic. K8s scales the pods; the queue distributes the work.

Visibility Timeout

When a consumer picks up a message from the queue, it has a visibility timeout: a window during which the message is invisible to other consumers. If the consumer processes the message and acknowledges it, the message is removed from the queue. If the consumer dies before acknowledging (pod killed, OOM, crash), the visibility timeout expires and the message becomes visible again for another consumer to pick up.

This is the mechanism that ensures work is not lost during pod restarts. The request re- enters the queue and is processed by a healthy replica. The idempotency key in the WorkRequest ensures that if the original consumer partially processed the request before dying, the replacement consumer detects the duplicate and either skips it (if the result was already stored) or re-executes it safely.

When WebSocket is Enough

Not every cross-ensemble interaction needs durable transport. For quick tool calls between co-located ensembles that are both healthy, direct WebSocket is faster and simpler.

The `NetworkTool` (shared tool call) defaults to WebSocket transport with a fallback: if the WebSocket connection is down, queue the request via the durable transport and wait. The `NetworkTask` (shared task delegation) defaults to durable transport because task executions are long-running and more vulnerable to pod restarts.

These defaults are configurable. A user who runs everything on a single machine during development can use WebSocket for everything. A user in production with strict reliability requirements can force durable transport for all interactions.

The State of the Art

The split between real-time and durable transport is not novel in distributed systems. It is the standard architecture for any system that needs both low-latency event streaming and reliable work processing:

- Trading systems: market data over UDP multicast (real-time), order execution over reliable messaging (durable)
- Chat applications: message display over WebSocket (real-time), message persistence in a database (durable)
- IoT platforms: telemetry over MQTT (real-time), command delivery over cloud queues (durable)

What is novel is applying this pattern to AI agent systems, where the "work" is LLM task execution and the "events" are agent lifecycle callbacks. The Ensemble Network does not invent new transport

infrastructure. It leverages existing, battle-tested infrastructure (Redis Streams, Kafka, WebSocket) and provides a clean abstraction over it through the Transport SPI.

Chapter 7: Human Participation

The Manager Who Goes Home

In every human-in-the-loop AI system built today, the human is a gatekeeper. The system runs a task, produces output, pauses, and presents the output to a human for approval. The human reviews it, clicks "Approve" or "Edit," and the system continues. If no human is present, the system waits. If the wait exceeds a timeout, the system either auto-approves (risky) or fails (wasteful).

This model treats the human as a **controller** – a required component in the execution pipeline. Without the controller, the pipeline stalls.

A hotel does not work this way. The general manager is not a controller. She is a **participant**. She joins the operation when she chooses, observes the current state, gives direction, handles things that need her authority, and leaves when she chooses. The hotel does not stall when she is absent. It runs autonomously. Her presence makes it better, but her absence does not make it stop.

The Ensemble Network implements this model. Humans are participants in the network, not controllers of it.

The Interaction Spectrum

Human interaction with the ensemble network exists on a spectrum from fully autonomous to fully gated. The level is configurable per task, per ensemble, or at the network level.

Autonomous

No human involvement at all. The ensemble handles the task entirely on its own.

Housekeeping cleans rooms after checkout. No approval needed. No notification. The process runs and the result is recorded in the audit trail.

This is the default. If no review gate is configured and no directives are active, the ensemble operates autonomously.

Advisory

A human provides guidance that influences future task executions. The guidance is non-blocking: the ensemble does not pause to wait for it. The human sends it when they think of it.

"Guest in 801 is a VIP, prioritize all their requests." This is a **directive**. It is injected into the ensemble's context and influences how agents reason about subsequent tasks. The agents see the

directive as additional context in their prompts.

```
{
  "type": "directive",
  "to": "room-service",
  "from": "manager:human",
  "content": "Guest in 801 is VIP, prioritize all their requests"
}
```

Directives are stored by the ensemble and injected as context into task executions until they expire (optional TTL) or are superseded by a newer directive.

Notifiable

The ensemble alerts a human about something noteworthy but does not wait for a response. The ensemble proceeds with its best-effort handling.

"Water leak detected in room 305, maintenance dispatched." The maintenance ensemble handles the emergency. It also sends a notification to the manager. The manager sees it on the dashboard (if connected) or in the notification log (when she connects later). She does not need to do anything – the ensemble handled it. But she is informed.

Approvable

The ensemble produces output and presents it for human review. If a human is available, they approve, edit, or reject. If no human is available within the timeout, the system applies a default action (usually auto-approve).

This is the existing v2.1.0 review gate model. A task pauses after producing output. The browser dashboard shows the review panel with Approve, Edit, and Exit Early buttons. A countdown timer shows the remaining timeout. When the timer expires, the configured `onTimeout` action (CONTINUE or EXIT_EARLY) is applied automatically.

```
Task.builder()
    .description("Draft marketing email")
    .review(Review.builder()
        .prompt("Review the draft before sending")
        .timeout(Duration.ofMinutes(5))
        .onTimeout(OnTimeoutAction.CONTINUE)
        .build())
    .build();
```

Approvable tasks degrade gracefully: if nobody is watching, the work still gets done.

Gated

The task cannot proceed without specific human authorization. There is no timeout. The task waits indefinitely for a qualified human to connect and make a decision.

Opening the hotel safe requires the manager. Not the receptionist, not the bell staff. The manager specifically. If the manager is not present, the safe stays closed. When the manager arrives, she sees the pending authorization request, approves it, and the task proceeds.

```
Task.builder()
    .description("Open the hotel safe for cash reconciliation")
    .review(Review.builder()
        .prompt("Manager authorization required to open the safe")
        .requiredRole("manager")
        .timeout(Duration.ZERO)  // no timeout -- wait indefinitely
        .build())
    .build();
```

The `requiredRole` field is the key addition. It restricts who can make the decision. A human connected to the dashboard with the "receptionist" role sees the pending review but cannot act on it – it is grayed out. When a human with the "manager" role connects, the review lights up and becomes actionable.

Out-of-Band Notification

For gated tasks, the ensemble should not silently wait. It should actively seek a qualified human:

```
Review.builder()
    .prompt("Manager authorization required")
    .requiredRole("manager")
    .timeout(Duration.ZERO)
    .notifyVia(ReviewNotifier.slack("#ops-channel"))
    .build()
```

When the gated review fires and no qualified human is connected, the ensemble: 1. Queues the review 2. Sends a Slack notification: "Review pending: Manager authorization required to open the hotel safe. Connect to approve: https://dashboard.hotel.example/network" 3. Continues waiting 4. When the manager connects (maybe from home on their phone), they see the pending review 5. They approve, and the task resumes

The ensemble did not stall. Other tasks continued processing. Only the specific gated task waited. The notification brought the right human to the right decision point.

Directives: Non-Blocking Guidance

Directives are the most common form of human interaction in a running network. They are the verbal instructions a manager gives while walking around the hotel.

Context Directives

Context directives add information that agents incorporate into their reasoning:

"The heating system is being serviced today. Route HVAC complaints to the service team, not our maintenance."

This is injected into the maintenance ensemble's context. When a maintenance agent receives an HVAC complaint, the directive appears in its prompt as additional context. The agent reasons about it and routes the complaint accordingly.

Context directives have an optional TTL. "VIP in 801 until Sunday" expires automatically. Without a TTL, the directive persists until explicitly removed or superseded.

Control Plane Directives

Control plane directives change ensemble behavior at runtime:

Model tier switching: "Switch the kitchen ensemble to the fallback (cheaper) LLM."

```
{
  "type": "directive",
  "to": "kitchen",
  "action": "SET_MODEL_TIER",
  "value": "FALLBACK"
}
```

The ensemble's subsequent tasks use the fallback model. In-flight tasks continue with their original model. This is for cost management: when the token spend is high, switch non-critical ensembles to cheaper models.

Profile application: "Apply the sporting-event-weekend profile."

```
{
  "type": "directive",
  "to": "*",
  "action": "APPLY_PROFILE",
  "value": "sporting-event-weekend"
}
```

This adjusts capacity for all ensembles in the network. Chapter 5 covers profiles in detail.

Who Can Send Directives?

Directives can come from: - A human via the dashboard - An automated policy rule (e.g., "when monthly token spend exceeds $500, switch to fallback models") - Another ensemble (a "management" ensemble could issue directives to "operational" ensembles)

The framework does not distinguish between these sources. A directive is a directive. The `from` field in the message records the source for the audit trail.

Late-Join: Catching Humans Up

The existing v2.1.0 late-join mechanism works beautifully for the network model.

When a human connects to the dashboard, the server sends a `hello` message containing: - The current state of all ensembles the human has access to - The current snapshot of all events since the last ensemble restart - Any pending review gates awaiting human decision

The human is instantly up to date. They do not need to have been watching continuously. They connect, see the current state, and start participating.

When the human disconnects, nothing changes for the ensembles. Events continue to be recorded. When the human reconnects later – maybe the next morning – they get the full snapshot again and are caught up in seconds.

This is the hotel manager arriving at 8am. Within minutes, the dashboard shows her everything that happened overnight. She does not need to have been present. The state is there. The pending decisions are there. She catches up and starts her day.

Role-Based Access

Not all humans see the same things.

The general manager sees all ensembles, all queues, all pending reviews, all metrics. She can send directives to any ensemble and approve any review gate.

The receptionist sees the front desk ensemble and related cross-department events (guest check-in/check-out notifications). She can send directives to the front desk and approve front-desk review gates. She cannot see the kitchen's internal operations or approve maintenance purchase orders.

The bell staff sees their specific task assignments. They can mark tasks complete but cannot send directives or approve reviews.

```
HumanRole manager = HumanRole.builder()
    .name("manager")
    .canObserve("*")
    .canDirect("*")
    .canApprove("*")
    .build();

HumanRole receptionist = HumanRole.builder()
    .name("receptionist")
    .canObserve("front-desk", "room-service")
    .canDirect("front-desk")
    .canApprove("front-desk")
    .subscribesTo("guest.checkout", "guest.checkin")
    .build();
```

Role definitions are configured on the network. The mapping from authenticated user to role is provided by the organization's identity provider (the framework does not implement authentication – it delegates to the existing infrastructure, typically K8s + service mesh + identity provider).

The Dashboard as a Portal

The dashboard is not a control room that must be staffed 24/7. It is a **portal** that humans step through when they want to participate.

Open the portal: see the current state, give direction, handle pending decisions. Close the portal: the system continues. Open it again tomorrow: catch up and participate again.

This model is natural for organizations where AI systems augment human work rather than replace it. The humans are still the decision-makers for critical matters. They are the authority for

compliance-sensitive actions. They provide strategic direction. But they do not need to babysit the system. It runs on its own.

Chapter 8: Observability

You Cannot Debug What You Cannot See

When a single ensemble runs in a single JVM, debugging is straightforward. The execution trace captures every LLM interaction, every tool call, every delegation. You open the trace in the visualizer and see exactly what happened.

When maintenance delegates to procurement, which delegates to logistics, which delegates to compliance – across four different Kubernetes pods in two different namespaces – the single-ensemble trace is not enough. Each ensemble has its own trace. The traces are disconnected. The question "why did the parts order take 3 hours?" requires manually correlating four separate traces.

The Ensemble Network addresses this with three observability layers: distributed tracing (cross-ensemble correlation), adaptive auditing (dynamic logging depth), and scaling metrics (operational visibility).

Distributed Tracing: The Thread Through the Maze

W3C Trace Context

Every cross-ensemble message carries W3C Trace Context headers. This is mandatory in the wire protocol – it is not optional, and it does not depend on whether the user has deployed an OpenTelemetry collector.

```
{
  "type": "task_request",
  "requestId": "maint-7721",
  "traceContext": {
    "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
    "tracestate": "agentensemble=maintenance"
  }
}
```

The `traceparent` header carries a trace ID (shared across all participants in the distributed trace) and a span ID (identifying this specific segment). When procurement receives the request, it creates a new span as a child of maintenance's span. The trace ID stays the same. When procurement delegates to logistics, the chain continues.

The result is a single distributed trace that spans the entire delegation chain:

```
Trace: 4bf92f3577b34da6a3ce929d0e0e4736
```

```
|
+-- maintenance: fix-boiler (23 min)
     |
     +-- procurement: purchase-parts (18 min)  [cross-ensemble]
          |
          +-- vendor-search tool (2 sec)
          +-- price-comparison tool (3 sec)
          +-- logistics: arrange-shipping (12 min)  [cross-ensemble]
               |
               +-- compliance: customs-clearance (8 min)  [cross-ensemble]
          +-- purchase-order tool (1 sec)
```

Open Jaeger (or Tempo, or Zipkin, or Datadog) and the full picture is there. Every cross-ensemble boundary is visible. Every tool call within each ensemble is a child span. The critical path – where the time actually went – is immediately obvious.

OpenTelemetry Integration

The optional `agentensemble-telemetry-opentelemetry` module creates OTel spans at framework boundaries:

Span name	When created
`ensemble.run`	Root span for an ensemble execution
`task.execute`	Per-task child span
`llm.call`	Per-LLM-interaction (with token count attributes)
`tool.execute`	Per-tool-call child span
`network.delegate`	CLIENT span when calling another ensemble
`network.handle`	SERVER span when receiving a cross-ensemble request

The CLIENT/SERVER span pair at ensemble boundaries is the standard OTel pattern for distributed service calls. The CLIENT span is created by the `NetworkTask`/`NetworkTool` in the calling ensemble. The SERVER span is created by the work request handler in the receiving ensemble. They share the same trace ID and have a parent-child relationship.

Spans carry domain-specific attributes:

```
agentensemble.ensemble.name = "maintenance"
agentensemble.task.description = "Fix boiler in building 2"
agentensemble.agent.role = "Senior Maintenance Engineer"
agentensemble.delegation.target = "procurement"
agentensemble.tokens.input = 1842
agentensemble.tokens.output = 523
```

ExecutionTrace Correlation

The existing `ExecutionTrace` (AgentEnsemble's native trace format) gains a `traceId` field linking it to the distributed OTel trace. The visualizer can show: "This execution trace is part of distributed

trace `4bf92f...`. View in Jaeger." Clicking the link opens the full distributed trace in the external trace viewer.

The `DelegationTrace` record (already used for agent-to-agent delegation within a single ensemble) is extended with a `crossEnsemble` flag and the target ensemble name for cross-ensemble delegations.

Adaptive Audit Trail

The Problem with Static Logging

Traditional logging uses static levels: DEBUG, INFO, WARN, ERROR. You set the level at deployment time and it stays there. In production, you run at INFO to keep log volume manageable. When something goes wrong, you wish you had DEBUG. You change the log level, reproduce the issue, and analyze the output. By the time you have the detailed logs, the original issue may have resolved itself.

For an always-on ensemble network, this is insufficient. The system runs continuously. The "something going wrong" may be a transient issue during a load spike at 3am. You cannot reproduce it on demand. You need the detailed logs at the moment the issue occurs, not after you notice it.

Leveled Auditing

The audit trail operates at four levels, conceptually similar to log levels but specific to ensemble network operations:

Level	What is recorded
`OFF`	Nothing (development/testing only)
`MINIMAL`	Cross-ensemble requests and responses: who called whom, when, result status
`STANDARD`	+ human decisions, review gate outcomes, priority changes, directives
`FULL`	+ LLM prompts and responses, tool inputs and outputs, memory reads and writes

The default is MINIMAL. This captures the essential operational flow without the volume of full LLM interaction logs.

Dynamic Rules

Audit level can escalate automatically based on conditions:

```
AuditPolicy policy = AuditPolicy.builder()
    .defaultLevel(AuditLevel.MINIMAL)

    // When the kitchen is under heavy load, capture more detail
    .rule(AuditRule.when("capacity_utilization > 0.8")
        .escalateTo(AuditLevel.STANDARD)
        .on("kitchen"))
```

```
    // When any task fails, capture everything for 10 minutes
    .rule(AuditRule.when("task_failed")
        .escalateTo(AuditLevel.FULL)
        .on("*")
        .duration(Duration.ofMinutes(10)))

    // When the manager is watching, capture more
    .rule(AuditRule.when("human_connected AND role == 'manager'")
        .escalateTo(AuditLevel.STANDARD)
        .on("*"))

    // During dinner rush, capture kitchen detail
    .rule(AuditRule.schedule("18:00-22:00")
        .escalateTo(AuditLevel.STANDARD)
        .on("kitchen", "room-service"))

    .build();
```

Four trigger types:

Metric-driven: Escalate when a metric crosses a threshold. "When queue depth exceeds 20" or "when capacity utilization exceeds 80%."

Event-driven: Escalate when a specific event occurs. "When a task fails" or "when a circuit breaker opens."

Schedule-driven: Escalate during known high-activity periods. "During dinner rush (6pm-10pm)" or "during the conference (Friday-Sunday)."

Human-triggered: Escalate when a human connects with a specific role. The manager connecting to the dashboard automatically increases audit detail – because if the manager is watching, she probably wants to see more.

All escalations are **temporary**. They revert when the triggering condition clears (metric drops, human disconnects) or when the specified duration expires (10 minutes after a task failure). This prevents audit log volume from growing unbounded.

Audit Sinks

The audit trail is written to a pluggable sink:

```
.auditSink(AuditSink.log())            // SLF4J structured logging
.auditSink(AuditSink.database(ds))     // JDBC append-only table
.auditSink(AuditSink.eventStream())    // Kafka topic for downstream consumers
```

All records are immutable, append-only, timestamped, and correlatable via the distributed trace ID. For regulated industries (finance, healthcare), the database sink provides the tamper-evident audit trail that compliance requires.

Scaling Metrics

Every ensemble exposes Prometheus/Micrometer metrics that serve two purposes: operational dashboards (Grafana) and auto-scaling triggers (K8s HPA).

Ensemble-Level Metrics

```
agentensemble_active_tasks{ensemble="kitchen"} 8
agentensemble_queued_requests{ensemble="kitchen"} 15
agentensemble_completed_tasks_total{ensemble="kitchen"} 1247
agentensemble_failed_tasks_total{ensemble="kitchen"} 3
agentensemble_capacity_utilization{ensemble="kitchen"} 0.85
agentensemble_max_concurrent{ensemble="kitchen"} 10
```

Cross-Ensemble Metrics

```
agentensemble_network_requests_total{from="room-service",to="kitchen",task="prepare-meal"} 89
agentensemble_network_request_duration_seconds{from="room-service",to="kitchen",quantile="0.95"
agentensemble_network_errors_total{from="room-service",to="kitchen",type="timeout"} 2
agentensemble_circuit_breaker_state{from="room-service",to="kitchen"} 0
```

Token Cost Metrics

```
agentensemble_tokens_total{ensemble="kitchen",direction="input"} 284000
agentensemble_tokens_total{ensemble="kitchen",direction="output"} 91000
agentensemble_estimated_cost_dollars{ensemble="kitchen"} 12.50
```

Token costs are tracked per ensemble and exposed as gauges. The framework does not enforce budgets in v3.0.0 – cost control is achieved via control plane directives (Chapter 7) that switch model tiers when spending exceeds thresholds. The metrics provide the visibility; the human (or automated policy) makes the decision.

The Observability Stack

The full observability picture for an ensemble network:

Layer	Tool	What it shows
Distributed traces	Jaeger / Tempo	Cross-ensemble delegation chains, latency breakdown
Metrics dashboards	Grafana	Queue depths, capacity utilization, error rates, token spend
Audit trail	Database / Kafka	Who did what, when, why (compliance)
Live dashboard	agentensemble-viz	Real-time ensemble status, pending reviews, task flow
Log aggregation	ELK / Loki	Framework and application logs, correlated by trace ID

Each layer serves a different audience and question: - **Traces**: "Why did this specific request take so long?" - **Metrics**: "Is the system healthy right now? Do we need to scale?" - **Audit**: "Who approved this purchase order and when?" - **Dashboard**: "What is happening right now across all departments?" - **Logs**: "What was the exact error message when that task failed?"

The framework provides integration points for all of these. It does not replace any of them. It generates the data (spans, metrics, audit records, events, logs) and lets the user's existing infrastructure handle storage, querying, and visualization.

Chapter 9: Error Handling and Resilience

Five Ways Things Go Wrong

In a single-process ensemble, errors are straightforward: a task fails, an exception is thrown, the workflow executor handles it. In a distributed network of ensembles, failures are more varied and more subtle.

Failure Mode A: Unreachable

The target ensemble does not exist. The pod is not running. DNS does not resolve. The Kubernetes deployment has zero replicas.

This is a **connection failure**. The `NetworkTask` cannot establish a WebSocket connection or the queue consumer finds no target queue. The failure is immediate and unambiguous.

Failure Mode B: Busy

The target ensemble is reachable but under heavy load. It accepts the request, queues it, and reports an estimated completion time that exceeds the caller's deadline.

This is a **capacity issue**. The request is accepted but the SLA cannot be met. Per the "bend, don't break" principle (Chapter 5), the provider does not reject – it reports honestly and lets the caller decide.

Failure Mode C: Crash Mid-Execution

The target ensemble accepted the request and started working on it. Five minutes into a 15-minute procurement process, the pod is killed (OOM, deployment rollout, node failure).

This is the most dangerous failure mode. Work has started but not completed. The calling ensemble is waiting for a result that may never arrive. With durable transport (Chapter 6), the request re-enters the queue and another pod picks it up. With WebSocket-only transport, the request is lost.

Failure Mode D: Business Error

The target ensemble processed the request and failed for a domain reason: "No vendors found for model X-420," "Budget exceeded for this cost center," "Allergen detected in the requested dish."

This is a **business error**. The system worked correctly; the domain constraint prevented completion. The caller needs to handle this differently from a transient failure – retrying will produce the same result.

Failure Mode E: Network Blip

The WebSocket connection drops for 5 seconds due to a transient network issue, then recovers. In-flight request state is unclear. Did the request reach the target? Was it processed? Was the response sent but not received?

This is a **transient failure**. With idempotency keys, the caller can safely retry. With durable transport, the request was in the queue all along and unaffected.

The Resilience Toolbox

The framework provides four mechanisms for handling failures. Each is configured per `NetworkTask` or `NetworkTool` call.

Timeouts

Every cross-ensemble call has a timeout. There is no such thing as "wait forever" in a distributed system (except for gated human reviews, which are a deliberate design choice).

```
NetworkTask.from("procurement", "purchase-parts")
    .timeout(Duration.ofMinutes(30))
    .connectTimeout(Duration.ofSeconds(10))
```

Two timeouts: - **Connect timeout**: How long to wait to establish the connection or submit to the queue. Short (seconds). Detects failure mode A (unreachable). - **Execution timeout**: How long to wait for the complete result after the request is accepted. Longer (minutes to hours). Detects failure mode C (crash mid-execution).

When a timeout fires, the `NetworkTask` throws a `NetworkTimeoutException`. The calling agent's tool call fails. The agent may retry, use a fallback, or report the failure.

Retry Policies

Retries address transient failures (modes A, C, E). They must not retry business errors (mode D) because retrying "no vendors found" will find no vendors again.

```
NetworkTask.from("procurement", "purchase-parts")
    .retryPolicy(RetryPolicy.builder()
        .maxAttempts(3)
        .backoff(Duration.ofSeconds(5), Duration.ofMinutes(1))
        .retryOn(ConnectionFailure.class, TimeoutException.class)
        .noRetryOn(TaskFailureResponse.class)
        .build())
```

The `.retryOn()` and `.noRetryOn()` clauses distinguish transient from business errors. `ConnectionFailure` and `TimeoutException` are transient – retry with exponential backoff. `TaskFailureResponse` is a business error – propagate immediately.

The idempotency key in the WorkRequest ensures that retries are safe. If the original request was partially processed before the failure, the receiver recognizes the duplicate `requestId` and returns the cached result (or continues from where it left off if the result is not yet available).

Circuit Breakers

If procurement has failed 5 times in the last minute, continuing to send requests wastes resources and adds latency. A circuit breaker stops trying until the target recovers.

```
NetworkTask.from("procurement", "purchase-parts")
    .circuitBreaker(CircuitBreaker.builder()
        .failureThreshold(5)
        .windowDuration(Duration.ofMinutes(1))
        .halfOpenAfter(Duration.ofMinutes(5))
        .build())
```

Three states: - **Closed** (normal): requests flow through. Failures are counted. - **Open** (tripped): all requests fail immediately without attempting the call. No network traffic. The calling agent gets an immediate `CircuitOpenException`. - **Half-open** (probing): after the cooldown period, one request is allowed through. If it succeeds, the circuit closes. If it fails, the circuit opens again.

Circuit breaker state is per caller-target pair. Room service's circuit breaker to kitchen is independent of maintenance's circuit breaker to kitchen.

The circuit breaker state is exposed as a metric (`agentensemble_circuit_breaker_state`) so operators can see which connections are healthy and which are tripped.

Fallback Strategies

When all retries are exhausted and the circuit breaker is open, the caller needs a plan B.

```
NetworkTask.from("procurement", "purchase-parts")
    .onFailure(Fallback.returnMessage(
        "Procurement is currently unavailable. The parts order could not be placed."))
```

Or route to an alternative provider:

```
NetworkTask.from("procurement", "purchase-parts")
    .onFailure(Fallback.delegateTo("procurement-backup", "purchase-parts"))
```

Or degrade gracefully:

```
NetworkTask.from("procurement", "purchase-parts")
    .onFailure(Fallback.custom(error -> {
        // Log the failure, notify the human, return partial result
        return "Parts order failed: " + error.getMessage()
            + ". Manual procurement required.";
    }))
```

The fallback result is returned to the calling agent as if it were a normal tool result. The agent incorporates it into its reasoning: "Procurement is unavailable, I should notify the building manager that the repair will be delayed."

Framework Handles Semantics, Infrastructure Handles Transport

A clean separation of concerns:

Concern	Owner	Rationale
Timeouts per call	Framework	The framework knows the call semantics
Retry with error distinction	Framework	The framework knows transient vs business
Circuit breaker state	Framework	The framework tracks per-target failure rates
Fallback strategies	Framework	The framework provides the alternative routing
TLS encryption	Infrastructure	K8s service mesh (Istio) handles mTLS
Connection pooling	Infrastructure	WebSocket/HTTP client handles pooling
Load balancing	Infrastructure	K8s Service provides round-robin
DNS resolution	Infrastructure	K8s CoreDNS handles service discovery
Health checks	Infrastructure	K8s probes use the framework's health endpoints
Auto-scaling	Infrastructure	K8s HPA uses the framework's metrics

The framework does not re-implement TLS, load balancing, or DNS. It does not need to. Kubernetes provides these at the infrastructure level. The framework focuses on the **semantic** layer: understanding what failed, deciding whether to retry, tracking circuit breaker state, and providing fallback behavior.

This separation means the same framework code works in local development (no Istio, no HPA, direct connections) and in production (full service mesh, auto-scaling, network policies). The framework's resilience logic is independent of the deployment environment.

The Durable Queue as a Resilience Primitive

The durable transport layer (Chapter 6) is itself a resilience mechanism. When work requests flow through a durable queue rather than a direct WebSocket connection:

- **Failure mode A** (unreachable): The request sits in the queue until the target starts. No retry needed – the queue handles the wait.
- **Failure mode C** (crash mid-execution): The visibility timeout expires and the request re-enters the queue. Another replica picks it up.
- **Failure mode E** (network blip): The queue is unaffected by transient network issues between pods. The request is already persisted.

The durable queue does not eliminate the need for timeouts and circuit breakers (the caller still needs to know when to give up waiting), but it absorbs many transient failures that would otherwise require explicit retry logic.

This is why the architecture recommends durable transport for production: it converts many failure modes from "the request is lost" to "the request is delayed." The "bend, don't break" principle applied to the transport layer itself.

Chapter 10: Testing, Simulation, and Chaos

You Cannot Set Fire to the Hotel

Testing a single ensemble is straightforward. You define tasks, provide a deterministic chat model, run the ensemble, and assert on the output. The entire execution happens in one process, one thread, one test method.

Testing a network of ensembles is a different problem. You have multiple services, multiple processes, multiple queues, multiple failure modes. You cannot spin up the entire hotel – all departments, all infrastructure, all connections – in a unit test. And even if you could, the most important questions are not "does it work when everything is healthy?" but "what happens when things go wrong?"

You cannot set fire to the hotel to test the fire response. But you can test the fire alarms, the sprinkler system, and the evacuation plan separately. And you can run a fire drill with real people.

This maps to three tiers of testing, each answering different questions at different costs.

Tier A: Component Tests – Test the Alarms

Component tests verify each ensemble in isolation. Cross-ensemble dependencies are replaced with test doubles. These tests are fast, cheap, and run in CI on every commit.

Stubs

A stub replaces a remote ensemble with a canned response:

```
NetworkTask procurementStub = NetworkTask.stub("procurement", "purchase-parts",
    "Ordered from SupplyCo, PO #4821, delivery Thursday");

Ensemble maintenance = Ensemble.builder()
    .chatLanguageModel(testModel)
    .task(Task.builder()
        .description("Fix boiler in building 2")
        .tools(procurementStub)
        .build())
    .build();
```

```
EnsembleOutput output = maintenance.run();
assertThat(output.getRaw()).contains("Thursday");
```

The maintenance ensemble does not know the procurement response is fake. Its agent calls the tool, gets the canned response, and continues reasoning. The test verifies that maintenance handles the procurement result correctly.

Recording Doubles

A recording double captures what was sent to a remote ensemble for assertion:

```
NetworkTask recorder = NetworkTask.recording("procurement", "purchase-parts");

// ... run the maintenance ensemble with the recorder ...

assertThat(recorder.lastRequest()).contains("valve", "X-420", "building 2");
assertThat(recorder.callCount()).isEqualTo(1);
```

This verifies that maintenance sends the right request to procurement without needing procurement to actually exist.

Contract Tests

Contract tests verify both sides of an inter-ensemble boundary independently. Each side tests against the shared contract without the other side being present.

Caller side (maintenance): "I send a valid procurement request."

```
@Test void maintenanceSendsValidProcurementRequest() {
    NetworkTask recorder = NetworkTask.recording("procurement", "purchase-parts");
    maintenanceEnsemble(recorder).run();
    assertThat(recorder.lastRequest()).contains("valve", "model");
}
```

Provider side (procurement): "Given a valid request, I produce a valid response."

```
@Test void procurementHandlesPartRequest() {
    EnsembleOutput output = procurementEnsemble.run(Map.of(
        "request", "Order replacement valve model X-420, urgent"));
    assertThat(output.getRaw()).containsPattern("PO #\\d+");
    assertThat(output.getRaw()).contains("delivery");
}
```

Neither test requires the other ensemble. Each tests its own side of the contract. If both pass, the integration should work. This is the same principle as Pact consumer-driven contract testing, adapted for natural language contracts.

Tier B: Simulation – Computer-Model the Evacuation

Simulation models the full network with simulated components: fake LLMs (fast, cheap, deterministic), time compression (run simulated hours in real minutes), and scenario definitions (load profiles,

failure injection, latency profiles).

Simulation answers questions that component tests cannot: - "How many kitchen replicas do we need for the conference?" - "What happens when procurement goes offline for 5 minutes during peak?" - "Where is the bottleneck when we have 200 concurrent check-ins?"

```
Simulation sim = Simulation.builder()
    .network(hotelNetwork)
    .scenario(Scenario.builder()
        .name("Conference peak load")
        .load("front-desk", LoadProfile.ramp(0, 200, Duration.ofMinutes(30)))
        .load("room-service", LoadProfile.steady(50))
        .failure("kitchen", FailureProfile.downAt(
            Duration.ofMinutes(15), Duration.ofMinutes(5)))
        .latency("procurement", LatencyProfile.multiply(3.0))
        .build())
    .chatModel(SimulationChatModel.fast())
    .timeCompression(60)  // 1 hour of simulated time = 1 minute of wall time
    .build();

SimulationResult result = sim.run();
```

SimulationChatModel

The `SimulationChatModel` is not a real LLM. It generates realistic-shaped responses without making API calls. It is configurable:

- **Response latency**: Simulates the time an LLM takes to respond (configurable per "model tier")
- **Response length**: Generates responses of realistic token count
- **Token counting**: Reports accurate token usage for cost estimation
- **Deterministic**: Same input produces same output (seeded randomness)

The simulation does not test LLM quality. It tests system behavior: queuing, routing, scaling, failure handling, capacity management. The LLM is a latency source and a token consumer, not an intelligence source, in simulation mode.

Scenario Definition

A scenario defines the conditions under which the simulation runs:

LoadProfile: How much work arrives at each ensemble over time. - `steady(n)`: N requests per minute, constant - `ramp(from, to, duration)`: Linearly increasing from `from` to `to` over `duration` - `spike(base, peak, at, duration)`: Spike from `base` to `peak` at time `at` for `duration`, then back to `base`

FailureProfile: When ensembles fail and recover. - `downAt(time, duration)`: Ensemble goes down at `time`, comes back after `duration`

LatencyProfile: How much slower an ensemble responds. - `fixed(duration)`: Add fixed latency to every response - `multiply(factor)`: Multiply normal latency by `factor`

SimulationResult

The simulation produces a structured report:

- **Bottlenecks**: Which ensembles hit capacity limits and when
- **Failure cascades**: How one ensemble's failure affected others (circuit breaker trips, fallback activations, queue buildup)
- **Capacity report**: Recommended replica counts for each ensemble under the simulated load
- **Token estimate**: Projected token consumption and cost at production LLM prices
- **Timeline**: Detailed event log of every request, response, failure, and recovery

The simulation can be visualized in the same dashboard used for live operation. The `/simulation` route in agentensemble-viz replays the simulation timeline, showing the same flow graphs and timeline views that the live dashboard shows.

Tier C: Chaos Engineering – Run the Drill with Real People

Simulation predicts behavior. Chaos engineering verifies it.

In a simulation, the LLMs are fake, the connections are simulated, and the failures are synthetic. In a chaos experiment, the ensembles are real (running in a staging environment), the LLMs are real (or realistic test doubles), and the failures are injected into the actual running system.

This is the fire drill. Real people (the operations team) watch the dashboard as faults are injected. They verify that the alarms fire, the failovers work, and the system recovers.

Built-In, Not Bolted On

The Ensemble Network's chaos engineering is built into the framework, not an external tool. This matters because the framework operates at the **application layer**, not the infrastructure layer.

External chaos tools (Chaos Monkey, Litmus, Gremlin) operate at the infrastructure level: kill a pod, partition a network, corrupt a disk. They cannot inject application-level faults: "Drop all `purchase-parts` requests to procurement but keep `check-inventory` working." "Simulate an LLM timeout on the kitchen ensemble but not on room service." "Degrade the kitchen's capacity to 50% without killing any pods."

The framework owns the network layer. It can inject precise, semantic faults that external tools cannot.

ChaosExperiment

```
ChaosExperiment experiment = ChaosExperiment.builder()
    .name("Kitchen outage during dinner rush")
    .against(stagingNetwork)

    // Fault schedule
    .at(Duration.ofMinutes(3),
        Fault.latency("procurement", Duration.ofSeconds(30)))
    .at(Duration.ofMinutes(5),
        Fault.kill("kitchen"))
```

```
    .at(Duration.ofMinutes(10),
        Fault.restore("kitchen"))

    // Expected behavior (assertions)
    .expect(Assertion.circuitBreakerOpens(
        "room-service", "kitchen", within(Duration.ofSeconds(30))))
    .expect(Assertion.fallbackActivated(
        "room-service", within(Duration.ofMinutes(1))))
    .expect(Assertion.noDataLoss())
    .expect(Assertion.allPendingRequestsResolve(
        within(Duration.ofMinutes(15))))

    .build();

ChaosReport report = experiment.run();
assertThat(report.passed()).isTrue();
```

Fault Types

Fault	What it does
`kill(ensemble)`	Stops the ensemble (simulates pod death)
`restore(ensemble)`	Restarts the ensemble
`latency(ensemble, duration)`	Adds latency to all responses
`dropMessages(ensemble, rate)`	Drops a percentage of messages
`degradeCapacity(ensemble, factor)`	Reduces concurrent task limit
`partialFailure(ensemble, task, rate)`	Fails a percentage of specific task requests

Assertion Types

Assertion	What it verifies
`circuitBreakerOpens(from, to, within)`	Circuit breaker trips within the expected time
`fallbackActivated(ensemble, within)`	Fallback strategy activates
`noDataLoss()`	All accepted requests eventually get a response
`allPendingRequestsResolve(within)`	No requests are stuck after recovery
`queueDrains(ensemble, within)`	Queue returns to normal depth

ChaosReport

The report includes: - **Pass/fail** for each assertion - **Timeline** of all faults injected and system responses - **Metrics snapshots** at key moments (before fault, during fault, after recovery) - **Human observations** (if operators were watching the dashboard during the drill and added notes)

The chaos experiment can be run as a CI test (automated, unattended) or as a game day exercise (operators watching the dashboard, practicing their response procedures).

The Testing Pyramid

```
          /\
         /  \  C. Chaos drills
        /    \     Real systems, real faults, real people
       /------\    Rare, expensive, high-confidence
      /        \
     /          \  B. Simulation
    /            \     Full network, simulated components
   /--------------\    Regular, moderate cost, good coverage
  /                \
 /                  \  A. Component tests
/                    \     Isolated ensembles, stubs, contracts
/--------------------\    Frequent, cheap, fast feedback
```

Each tier builds on the one below it. Component tests verify that each piece works. Simulation predicts how the pieces interact. Chaos engineering proves it.

Together, they answer the three questions: - Do the alarms work? (component tests) - What does the computer model predict? (simulation) - Does the drill go as planned? (chaos engineering)

You still cannot set fire to the hotel. But you can be confident that when the fire alarm goes off, the sprinklers activate, the exits are clear, and the staff know what to do.

Chapter 11: Shared State and Consistency

Two Kinds of Truth

A hotel maintains two fundamentally different kinds of information.

The first kind is **advisory context**: "Guest Smith prefers a quiet room on upper floors." "The boiler in building 2 has been unreliable this month." "The conference group tends to order room service late." This information is useful, imprecise, and tolerant of inconsistency. If two departments have slightly different notes about Guest Smith's preferences, no harm is done. The agents reading the notes can synthesize both perspectives.

The second kind is **authoritative state**: "Room 403 is assigned to Guest Smith." "We have 3 replacement valves in inventory." "PO #4821 has been submitted." This information must be accurate. If two departments disagree about who is in room 403, someone shows up to a room that is already occupied. If the inventory count is wrong, procurement orders parts that already exist – or fails to order parts that are needed.

The Ensemble Network recognizes this distinction and provides different consistency models for each.

Advisory Memory: Eventual Consistency

Shared memory scopes used for advisory context operate with eventual consistency and last-write-wins semantics. This is the default and covers the majority of cross-ensemble shared state.

```
network.sharedMemory("guest-preferences", SharedMemory.builder()
    .store(MemoryStore.embeddings(embeddingModel, store))
    .consistency(Consistency.EVENTUAL)
    .build());
```

When the front desk writes "Guest Smith likes extra pillows" and room service writes "Guest Smith always orders decaf," both entries are stored. Neither overwrites the other. When a task reads from the `guest-preferences` scope, both entries are retrieved (via semantic similarity or recency-based retrieval) and injected into the agent's prompt.

Concurrent writes do not conflict because they are additive. Memory entries are appended, not replaced. The LLM reads all available context and synthesizes it. This is the natural language

equivalent of eventual consistency: the agent gets a slightly stale or slightly redundant view, and it handles it gracefully because it is interpreting natural language, not parsing a strict schema.

Authoritative State: Choose Your Consistency

For state that must be accurate, the framework provides configurable consistency models per scope.

Distributed Locking

For exclusive access – only one ensemble can modify this state at a time:

```
network.sharedMemory("room-assignments", SharedMemory.builder()
    .store(MemoryStore.redis(client))
    .consistency(Consistency.LOCKED)
    .lockProvider(LockProvider.redis(client))
    .build());
```

When the front desk assigns a guest to a room, it acquires a distributed lock on the `room-assignments` scope, reads the current state, writes the new assignment, and releases the lock. While the lock is held, no other ensemble can modify room assignments.

The lock is implemented via the `LockProvider` SPI. The Redis implementation uses `SETNX` with an expiry (or the Redlock algorithm for multi-node Redis). Other implementations (ZooKeeper, database advisory locks) are pluggable.

Lock timeouts prevent deadlocks. If the ensemble holding the lock crashes without releasing it, the lock expires after the configured timeout and another ensemble can acquire it.

Optimistic Concurrency

For counters and quantities where locking is too heavy:

```
network.sharedMemory("inventory-count", SharedMemory.builder()
    .store(MemoryStore.redis(client))
    .consistency(Consistency.OPTIMISTIC)
    .build());
```

Optimistic concurrency uses compare-and-swap. The ensemble reads the current value (and its version), computes the new value, and writes with the version check. If another ensemble modified the value between the read and the write, the write fails and the ensemble retries with the updated value.

This is lighter than locking for high-contention scenarios. Multiple ensembles can read the inventory count concurrently. Only the write path requires conflict detection.

External Systems

For state managed by a database, booking system, or ERP:

```
network.sharedMemory("financial-ledger", SharedMemory.builder()
    .consistency(Consistency.EXTERNAL)
    .build());
```

The `EXTERNAL` consistency model means the framework does not manage the state at all. The ensemble's agents access the state through tools (database query tools, API tools) that connect directly to the external system. The external system provides its own consistency guarantees (ACID transactions, optimistic locking, etc.).

This is the right choice for systems of record. The hotel's financial ledger is not a memory scope – it is a database. Agents query it and update it through tools that respect the database's transactional semantics.

The Right Model for the Right Data

Data	Consistency	Why
Guest preferences	EVENTUAL	Advisory; LLM tolerates inconsistency
Interaction notes	EVENTUAL	Additive context; no conflicts
Room assignments	LOCKED	Exclusive; double-booking is unacceptable
Inventory counts	OPTIMISTIC	Concurrent reads, occasional writes
Financial records	EXTERNAL	System of record with ACID guarantees
Purchase orders	EXTERNAL	External procurement system

The key insight: **most shared state in an AI agent network is advisory context**, not authoritative data. The LLM is the consistency model for advisory context – it reads whatever is available and makes sense of it. Only the genuinely authoritative data needs stronger consistency, and that data usually lives in an external system that already provides it.

The framework does not try to build a distributed database. It provides a spectrum of consistency options and lets the user choose per scope. For most scopes, eventual consistency is not just acceptable – it is preferable, because it is simpler, faster, and more resilient.

Chapter 12: Federation – The Hotel Chain

Beyond a Single Cluster

A single hotel is a network of departments. A hotel chain is a network of hotels.

When Hotel A downtown is hosting a 500-person conference and every department is at capacity, Hotel B at the airport – which is under renovation and running at 30% occupancy – has idle kitchen capacity, idle housekeeping capacity, and maintenance staff with time to spare.

In the physical world, you cannot move Hotel B's kitchen to Hotel A. But in a distributed AI system, you can. If the kitchen ensemble is a service that accepts work requests over a network, it does not matter which cluster the request comes from. Hotel A's room service can delegate "prepare-meal" to Hotel B's kitchen, and the guest in Hotel A gets their sandwich.

This is **federation**: multiple independent realms (hotels) that share capabilities across boundaries.

Realms

A realm is a trust and discovery boundary. In Kubernetes terms, a realm maps to a namespace (or a cluster). Ensembles within a realm discover each other via K8s DNS and communicate directly.

```
Federation: "Hotel Chain"
  +-- Realm: hotel-downtown  (K8s namespace or cluster)
  |     +-- kitchen
  |     +-- room-service
  |     +-- maintenance
  |     +-- procurement
  |
  +-- Realm: hotel-airport
  |     +-- kitchen (idle capacity)
  |     +-- housekeeping (idle capacity)
  |     +-- maintenance (busy with renovation)
  |
  +-- Realm: hotel-beach
        +-- kitchen
        +-- room-service
        +-- maintenance
```

Within a realm, discovery is automatic (K8s DNS). Across realms, discovery requires configuration: which realms are in the federation, what gateway endpoints connect them, and what authentication/authorization governs cross-realm requests.

Capacity Advertisement

For cross-realm routing to work, ensembles must advertise their capacity – not just their capabilities but their current availability:

```
{
  "type": "capacity_update",
  "ensemble": "kitchen",
  "realm": "hotel-airport",
  "status": "available",
  "currentLoad": 0.2,
  "maxConcurrent": 10,
  "availableCapacity": 8,
  "shareable": true
}
```

The `shareable` flag is critical. It means: "I have capacity available for cross-realm requests." During normal operations, a hotel's kitchen might set `shareable: false` – its capacity is reserved for its own guests. During renovation (low occupancy), it sets `shareable: true` – spare capacity is offered to the federation.

Capacity updates are broadcast periodically (every 30 seconds, configurable). They are advisory, not commitments. By the time a cross-realm request arrives, the capacity may have changed. The receiving ensemble handles this gracefully: accept and queue (bend, don't break), or reject if at hard limits.

Routing Hierarchy

When an ensemble needs a capability (e.g., "prepare-meal"), the routing hierarchy is:

1. **Local**: Is there a provider in my ensemble? (agent delegation, existing v2.x)
2. **Realm**: Is there a provider in my realm? (same K8s namespace, cross-ensemble)
3. **Federation**: Is there a provider in another realm? (cross-namespace/cross-cluster)

Each level has its own routing strategy:

- **Local**: direct (no network)
- **Realm**: K8s DNS + direct WebSocket or queue
- **Federation**: gateway endpoints, potentially with higher latency and stricter authentication

The caller does not explicitly choose the level. The `NetworkTask`/`NetworkTool` implementation resolves the target at request time:

```
// The caller just says "kitchen"
NetworkTask.from("kitchen", "prepare-meal")
```

If a local kitchen ensemble is available, it handles the request. If the local kitchen is at capacity and a federated kitchen has spare capacity, the request routes to the federated provider. The caller's agent does not know or care where the meal is being prepared.

When to Federate

Federation is not for every scenario. It introduces cross-realm latency, authentication overhead, and operational complexity. It is valuable in specific situations:

Elastic overflow: The primary provider is at capacity during a peak event. The federation provides burst capacity from an idle realm.

Disaster recovery: The primary provider is down (datacenter outage, major failure). Another realm can serve requests while the primary recovers.

Geographic distribution: Work is routed to the nearest realm with capacity. A guest in the airport hotel gets their meal from the airport hotel's kitchen, not the downtown hotel's kitchen.

Specialization: One realm has a capability that others do not. The downtown hotel has a five-star restaurant kitchen with a pastry chef. The airport hotel does not. Cross-realm delegation to the specialized kitchen is the only way to fulfill the request.

Cross-Realm Authentication

Within a realm, authentication is handled by the K8s infrastructure (service mesh mTLS, network policies). Ensembles in the same namespace trust each other.

Across realms, additional authentication is required. The framework defines an SPI for cross-realm authentication but does not implement specific mechanisms. The expected production deployment uses:

- **Service mesh federation** (Istio multi-cluster): mTLS certificates issued by a shared CA, automatically verified at the gateway
- **API tokens**: pre-shared tokens validated at the federation gateway
- **OAuth/OIDC**: machine-to-machine tokens issued by a shared identity provider

The WorkRequest envelope carries authentication credentials in a header (never in the natural language context). The receiving realm's gateway validates credentials before forwarding the request to the target ensemble.

The Hotel Chain as a Design Pattern

The federation model is not just a technical feature. It is an organizational design pattern.

A hotel chain has: - **Shared brand standards**: All hotels offer the same core capabilities (rooms, restaurant, maintenance) with the same quality expectations. - **Local autonomy**: Each hotel operates independently. The downtown hotel does not need permission from headquarters to clean a room. - **Mutual support**: Hotels help each other during peaks, outages, or special events. - **Centralized policies**: Pricing, HR, and financial policies are set at the chain level.

This maps to the ensemble network: - **Shared capability contracts**: All realms implement the same shared task names with compatible natural language contracts. - **Local autonomy**: Each realm operates independently. Its ensembles handle their own work without coordinating with other realms. - **Federated capacity sharing**: Realms share spare capacity during peaks. - **Network-level policies**: Audit policies, operational profiles, and cost management rules can be set at the federation level.

The federation is not a hierarchical control structure. It is a peer network where independent realms choose to cooperate. Each realm can operate in isolation; the federation adds resilience and flexibility without creating dependency.

Chapter 13: Discovery and Evolution

Finding What You Need

In a small hotel, everyone knows everyone. Room service knows the kitchen is down the hall. Maintenance knows procurement is on the second floor. The organizational chart is small enough to memorize.

In a hotel chain with dozens of properties, you cannot memorize every department in every hotel. You need a directory. And in a dynamic environment – where new departments open, old ones close, and capabilities shift with seasonal demand – you need a directory that updates itself.

The Ensemble Network provides three levels of capability discovery, from static to fully dynamic.

Level 1: Static Configuration

The simplest approach. You know your ensembles at deployment time and configure connections explicitly:

```
NetworkTask.from("kitchen", "prepare-meal")
NetworkTool.from("kitchen", "check-inventory")
NetworkTask.from("procurement", "purchase-parts")
```

The ensemble names resolve via K8s DNS (`kitchen.hotel-downtown.svc.cluster.local`). No discovery protocol needed. This is sufficient for most single-realm deployments where the organizational structure is known and stable.

Level 2: Capability Registration

When ensembles start, they publish their shared capabilities via the capability handshake protocol. Other ensembles can query the network for providers of a specific capability:

```
// "Who provides check-inventory?"
NetworkTool.discover("check-inventory")
```

The framework resolves the query at call time: it looks up which ensemble registered the "check-inventory" shared tool and routes the request there.

Capability registration happens automatically when an ensemble starts (via the `ensemble_register` wire protocol message). De-registration happens when the ensemble shuts down or when its K8s readiness probe fails.

This level is useful when: - Multiple ensembles might provide the same capability (load-balancing across providers) - The specific provider does not matter to the caller (any kitchen will do) - Ensembles come and go dynamically (autoscaling, rolling deployments)

Level 3: Dynamic Tool Catalog

The most powerful form of discovery. An ensemble's available tools are not fixed at build time – they are discovered from the network at execution time:

```
Task.builder()
    .description("Handle any guest request")
    .tools(
        guestDatabaseTool,                  // fixed local tool
        NetworkToolCatalog.all(),            // all tools on the network
        NetworkToolCatalog.tagged("food"))   // tools tagged "food"
    .build();
```

`NetworkToolCatalog.all()` queries the network registry at task execution time (not at build time) and returns tool descriptions for all shared tools and tasks across all registered ensembles. The agent's LLM sees them as available tools and decides which to call based on the task at hand.

This means: **a new ensemble comes online and its tools are immediately available to every agent on the network**, without restarting or reconfiguring anything.

Tags

Shared capabilities can be tagged for filtered discovery:

```
.shareTask("prepare-meal", Task.builder()
    .description("Prepare a meal as specified")
    .build(),
    Tags.of("food", "kitchen"))

.shareTool("check-inventory", inventoryTool,
    Tags.of("food", "inventory"))
```

Tags allow callers to scope discovery. `NetworkToolCatalog.tagged("food")` returns only food-related capabilities. This prevents an agent from being overwhelmed with irrelevant tools from every ensemble on the network.

Why Natural Language Contracts Eliminate Schema Versioning

In a traditional microservice architecture, API evolution is a major engineering challenge. Changing a field name in a JSON response breaks all clients. Adding a required field breaks all clients. Teams spend significant effort on API versioning, backward compatibility, deprecation policies, and migration guides.

The Ensemble Network sidesteps this entirely.

The contract between ensembles is natural language. When maintenance asks procurement to "order

a replacement valve," the contract is not a typed schema – it is a description of what is needed and what the response should contain. The LLM on each side interprets the natural language.

If procurement changes how it phrases its output – maybe it used to say "Delivery expected Thursday" and now says "Estimated arrival: Thursday PM" – maintenance's agent interprets the new phrasing without any code change. The LLM is the compatibility layer.

When Does This Break?

Natural language contracts have limits:

Semantic changes: If "purchase-parts" changes from "buy parts" to "request quotes for parts" (different behavior, same name), callers expecting purchased parts will be confused. The convention is to use a new task name for semantically different behavior. "Purchase- parts" and "request-quotes" are different tasks.

Structured output changes: If a caller expects a `PurchaseOrder.class` structured output and procurement changes the field names, Jackson deserialization may fail. The mitigation is `@JsonIgnoreProperties(ignoreUnknown = true)` for forward compatibility, and treating structured output as optional (the natural language output is always available).

Capability removal: If procurement stops offering "purchase-parts" entirely, callers get a "capability not found" error. This is handled by the error handling mechanisms in Chapter 9 (fallback, circuit breaker).

For the vast majority of changes – rewording, adding detail, changing internal process, improving output quality – natural language contracts are completely transparent. This is a genuine advantage of AI-native systems over traditional typed-API systems.

Evolution Without Coordination

The most powerful consequence of natural language contracts and dynamic discovery is that ensembles can evolve independently.

The kitchen can: - Change its internal process (switch from sequential to parallel meal preparation) - Upgrade its LLM (from GPT-4 to Claude) - Add new shared tools (expose a "daily-specials" tool) - Improve its output quality (more detailed preparation notes)

None of these changes require any modification to room service, maintenance, or any other ensemble. The kitchen's callers send the same natural language requests and interpret the natural language responses. Internal changes are invisible. New capabilities appear automatically via the capability registration protocol.

This is the microservice independence property – services evolve independently as long as they respect their contracts – but with a much weaker contract requirement. Natural language is a far more flexible contract than a typed JSON schema.

The hotel does not shut down for renovations of the kitchen. The kitchen renovates while continuing to serve meals. The rest of the hotel does not notice.

Chapter 14: Infrastructure-Native Deployment

Let Kubernetes Do Its Job

A common mistake in framework design is rebuilding infrastructure capabilities that already exist. Custom service registries when K8s DNS is available. Custom health check systems when K8s probes are available. Custom auto-scalers when HPA is available. Custom load balancers when K8s Services are available.

The Ensemble Network does not make this mistake. It is **infrastructure-native**: designed from the ground up to run on Kubernetes, leveraging K8s for everything K8s is good at and providing only what K8s does not.

Concern	K8s provides	Framework provides
Service discovery	DNS (`kitchen.hotel-ns.svc.cluster.local`)	Nothing – use K8s DNS
Health management	Liveness/readiness probes	Health endpoints on the ensemble server
Load balancing	K8s Service (round-robin)	Nothing
Auto-scaling	HPA with custom metrics	Metrics (queue depth, capacity utilization)
Compartmentalization	Namespaces	Realm concept maps to namespace
Network security	Network policies + service mesh mTLS	SPI for auth (delegates to infra)
Secret management	K8s Secrets, env vars	Nothing – secrets stay in K8s
Graceful lifecycle	SIGTERM, terminationGracePeriod	Drain mode, preStop hook endpoint

Each Ensemble Is a Deployment

The deployment unit is simple: one Kubernetes Deployment per ensemble, fronted by a Service.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
```

```yaml
  name: kitchen
  namespace: hotel-downtown
  labels:
    app: kitchen
    agentensemble.io/role: ensemble
spec:
  replicas: 2
  selector:
    matchLabels:
      app: kitchen
  template:
    metadata:
      labels:
        app: kitchen
    spec:
      terminationGracePeriodSeconds: 300
      containers:
      - name: kitchen
        image: hotel/kitchen-ensemble:latest
        ports:
        - name: ws
          containerPort: 7329
        env:
        - name: ENSEMBLE_NAME
          value: "kitchen"
        - name: REDIS_URL
          valueFrom:
            secretKeyRef:
              name: redis-credentials
              key: url
        - name: OPENAI_API_KEY
          valueFrom:
            secretKeyRef:
              name: llm-credentials
              key: openai-key
        resources:
          requests:
            memory: "512Mi"
            cpu: "250m"
          limits:
            memory: "2Gi"
            cpu: "2000m"
        livenessProbe:
          httpGet:
            path: /api/health/live
            port: 7329
          initialDelaySeconds: 10
```

```yaml
          periodSeconds: 15
        readinessProbe:
          httpGet:
            path: /api/health/ready
            port: 7329
          initialDelaySeconds: 5
          periodSeconds: 5
        lifecycle:
          preStop:
            httpGet:
              path: /api/lifecycle/drain
              port: 7329
```

Health Endpoints

The ensemble provides three HTTP endpoints for Kubernetes integration:

`GET /api/health/live` – Liveness probe. Returns 200 if the JVM is alive and the ensemble process has not deadlocked. If this fails, K8s restarts the pod.

`GET /api/health/ready` – Readiness probe. Returns 200 only when the ensemble is in the READY state (not STARTING, not DRAINING). When this returns non-200, K8s removes the pod from the Service's endpoint list: new connections and requests are not routed to it.

`POST /api/lifecycle/drain` – Triggers the DRAINING state. Called by the `preStop` hook when K8s is about to terminate the pod. The ensemble stops accepting new work, finishes in-flight tasks, and delivers results before the pod exits.

Graceful Shutdown Sequence

1. K8s decides to terminate the pod (scale-down, rolling update, node drain)
2. K8s calls the `preStop` hook: `POST /api/lifecycle/drain`
3. Ensemble transitions to DRAINING state
4. Readiness probe returns non-200; K8s stops routing new connections
5. Ensemble stops pulling from the durable queue
6. Scheduled tasks stop running
7. In-flight tasks continue until completion or drain timeout
8. Results for completed tasks are delivered
9. K8s sends SIGTERM
10. Ensemble stops (STOPPED state)
11. If drain timeout has not expired, K8s waits (`terminationGracePeriodSeconds`)
12. If drain timeout expires, K8s sends SIGKILL

`terminationGracePeriodSeconds` must be at least as long as the ensemble's `drainTimeout`. For ensembles that process long tasks (procurement, research), this may be several minutes.

The Service

```yaml
apiVersion: v1
kind: Service
```

```yaml
metadata:
  name: kitchen
  namespace: hotel-downtown
spec:
  selector:
    app: kitchen
  ports:
  - name: ws
    port: 7329
    targetPort: 7329
```

Other ensembles connect to `ws://kitchen:7329/ws` (K8s DNS resolves within the namespace) or `ws://kitchen.hotel-downtown:7329/ws` (cross-namespace). K8s Service load-balances across healthy replicas.

Auto-Scaling

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: kitchen-hpa
  namespace: hotel-downtown
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: kitchen
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Pods
    pods:
      metric:
        name: agentensemble_queued_requests
      target:
        type: AverageValue
        averageValue: 5
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
      - type: Pods
        value: 1
        periodSeconds: 60
```

The HPA watches `agentensemble_queued_requests` (exposed by the framework via Micrometer). When the average queue depth exceeds 5 per pod, HPA adds replicas. When load drops, it removes replicas (with a stabilization window to avoid flapping).

The custom metric reaches Kubernetes via the Prometheus adapter (or KEDA, or any metrics pipeline that feeds the K8s custom metrics API). The framework exposes the metric; the infrastructure handles the plumbing.

What the Framework Provides for K8s

The framework provides: 1. **Health endpoints** (`/api/health/live`, `/api/health/ready`) 2. **Drain endpoint** (`/api/lifecycle/drain`) 3. **Prometheus metrics** (queue depth, active tasks, capacity utilization, token counts) 4. **Lifecycle state machine** (STARTING -> READY -> DRAINING -> STOPPED) 5. **Stateless execution** (external queue + external result store = pods are disposable)

The framework does not provide: - Helm charts (organizational choice) - Terraform/Pulumi definitions (infrastructure-as-code choice) - CI/CD pipelines (tooling choice) - Monitoring dashboards (Grafana templates are a nice-to-have but not framework scope)

The framework is a good Kubernetes citizen. It follows the established patterns (probes, graceful shutdown, custom metrics, stateless pods). Operators who know K8s can deploy ensembles using the same tools and practices they use for any other K8s workload.

Local Development

For development, none of the K8s infrastructure is needed. A developer runs ensembles directly:

```
Ensemble kitchen = Ensemble.builder()
    .name("kitchen")
    .chatLanguageModel(model)
    .shareTask("prepare-meal", mealTask)
    .build();

kitchen.start(7329);  // localhost:7329, simple WebSocket transport
```

Simple transport (in-process queues, direct WebSocket) is the default. No Redis, no Kafka, no K8s. Two ensembles running in the same JVM (or two JVMs on the same machine) can communicate directly.

The transition from local development to K8s production is a configuration change (transport mode, Redis URL, K8s manifests), not a code change. The ensemble code is the same in both environments.

Chapter 15: The Road Ahead

What We Have Built

The Ensemble Network, as described in this book, is an architecture for distributed, autonomous, human-augmented AI systems. The core pieces:

- **Long-running ensembles** that operate as services, not scripts
- **Cross-ensemble delegation** of complex, multi-step tasks
- **The WorkRequest protocol** with standardized envelope, priority, deadlines, and pluggable delivery
- **"Bend, don't break" capacity management** with caller-side SLAs and operational profiles
- **Durable transport** separating real-time events from reliable work delivery
- **Human participation** on a spectrum from autonomous to gated, with role-based access
- **Distributed tracing** via OpenTelemetry with adaptive audit trails
- **Resilience** through timeouts, retries, circuit breakers, and fallbacks
- **Testing** at three tiers: component, simulation, and chaos engineering
- **Shared state** with per-scope consistency models
- **Federation** across clusters and namespaces
- **Dynamic discovery** with natural language contracts that eliminate schema versioning
- **Infrastructure-native deployment** on Kubernetes

This is a foundation. Like any foundation, it exists to support what is built on top of it. Here are the directions that this architecture naturally leads toward.

Semantic Capability Matching

The current discovery model is name-based: "find the ensemble that provides `check- inventory`." This requires the caller to know the exact name of the capability.

Semantic matching uses embeddings to match work requests to capabilities by meaning:

```
// Instead of exact name match:
NetworkTool.from("kitchen", "check-inventory")

// Semantic match:
NetworkTool.discover("Is wagyu beef available in the kitchen right now?")
```

The framework embeds the query and compares it against the embedded descriptions of all registered capabilities. The closest match is selected. This enables a more natural interaction model where agents describe what they need rather than knowing the exact API.

This is analogous to asking a hotel concierge "Can I get a steak tonight?" versus knowing that the kitchen's capability registry includes item code `FOOD-BEEF-WAGYU`. The concierge model is more natural and more resilient to changes in the provider's internal naming.

Autonomous Ensemble Spawning

When the network detects sustained demand for a capability that no ensemble currently provides, it could automatically deploy a new ensemble to serve it.

The conference generates hundreds of requests for "translation-japanese." No ensemble provides this. The network detects the unserved capability, generates an ensemble configuration from the capability description, deploys it to K8s, and registers it. The next translation request is served.

This requires: - Demand detection (tracking failed capability queries) - Ensemble generation (synthesizing an ensemble configuration from a capability description) - Automated deployment (K8s API access to create Deployments and Services) - Lifecycle management (scale down or remove when demand drops)

This is speculative but architecturally sound: the framework already has agent synthesis (generating agents from task descriptions) and K8s deployment support. Extending synthesis from "generate an agent for a task" to "generate an ensemble for a capability" is a natural progression.

Cross-Organization Federation

The federation model described in Chapter 12 assumes all realms are within a single organization. Cross-organization federation – where ensembles owned by different companies communicate through a trust-brokered protocol – opens new possibilities.

A hotel chain partners with a local tour operator. The hotel's concierge ensemble delegates "book city tour" to the tour operator's booking ensemble. The tour operator's ensemble runs in a completely separate infrastructure, owned by a different company, with different security policies.

This requires: - Trust brokering (mutual authentication between organizations) - Data classification (what information can cross organizational boundaries) - Billing and metering (cross-organization usage tracking) - SLA enforcement (contractual commitments backed by technical mechanisms)

This is the AI equivalent of B2B API integration, but with natural language contracts and task-level delegation instead of REST APIs and JSON schemas.

Formal Verification of Delegation Chains

As networks grow, the risk of delegation cycles (A delegates to B, B delegates to C, C delegates to A) or delegation deadlocks (A waits for B, B waits for A) increases. Static analysis could prove that a given network configuration cannot enter these states.

The inputs are: - Each ensemble's shared capabilities (what it offers) - Each ensemble's consumed capabilities (what it calls) - The delegation depth limits

A graph analysis can detect potential cycles. A deadlock detector can identify mutual wait conditions. These checks could run at deployment time (blocking a configuration that creates a cycle) or at runtime (detecting and breaking cycles when they occur).

Cost-Optimal Routing

When multiple ensembles provide the same capability at different costs (different LLM providers, different model tiers, different regions with different pricing), the routing layer could minimize cost while meeting the caller's deadline.

"I need `prepare-meal` within 30 minutes. Kitchen A (GPT-4, $0.03/request) can do it in 10 minutes. Kitchen B (GPT-4 Mini, $0.003/request) can do it in 25 minutes." Cost-optimal routing selects Kitchen B: it meets the deadline at 10x lower cost.

This requires: - Per-ensemble cost metrics (cost per request, cost per token) - Deadline-aware routing (select the cheapest provider that can meet the deadline) - Cost budgets per ensemble or per network (monthly spending caps)

Ensemble Learning and Adaptation

Long-running ensembles accumulate experience. A kitchen ensemble that has processed 10,000 meal orders has "seen" a wide variety of requests, edge cases, and failure modes. This experience is currently stored in memory scopes but is not used to improve the ensemble itself.

Adaptation could take several forms: - **Prompt optimization**: Analyzing past task executions to identify the most effective prompt structures and agent personas - **Tool selection learning**: Learning which tools are most effective for which types of requests (reducing unnecessary tool calls) - **Priority prediction**: Learning to estimate request complexity and queue wait times more accurately based on past performance - **Failure prediction**: Identifying patterns that precede failures (load spikes, specific request patterns) and proactively scaling or routing around them

The Bigger Picture

The Ensemble Network architecture is, at its core, a claim about how AI systems should be structured for production use. The claim has several parts:

AI agents should be organized into autonomous groups (ensembles), not thrown into a single monolithic process. Each group has its own domain, its own expertise, its own lifecycle.

Groups should communicate through natural language delegation, not typed API contracts. The LLM is the compatibility layer. Natural language contracts are more resilient, more flexible, and more natural for AI-native systems than JSON schemas.

Work should be queued, not rejected. LLM tasks are inherently async. The caller sets deadlines; the provider reports honestly. "Bend, don't break."

Humans should be participants, not controllers. The system runs autonomously. Humans add value when present but are not required for operation. Critical decisions can require specific human authority.

Infrastructure should be leveraged, not rebuilt. Kubernetes provides service discovery, scaling, health management, and security. The framework provides what K8s cannot: semantic routing, natural language contracts, adaptive observability, and built-in chaos engineering.

These claims are not theoretical. They emerge from the observation that real organizations – hotels, hospitals, logistics companies, military units – already operate this way. The Ensemble Network

makes this operational model available to AI systems.

The hotel that never sleeps is not a metaphor. It is an architecture. And it is ready to be built.

Appendix A: Wire Protocol Reference

All messages are UTF-8 JSON with a `type` discriminator field. This appendix catalogs every message type in the Ensemble Network wire protocol, extending the v2.1.0 protocol defined in design document 16.

Existing v2.1.0 Messages (Unchanged)

These messages are retained from the live dashboard protocol:

Type	Direction	Purpose
`hello`	Server -> Client	Late-join state snapshot
`ensemble_started`	Server -> Client	Ensemble run begins
`ensemble_completed`	Server -> Client	Ensemble run ends
`task_started`	Server -> Client	Task execution begins
`task_completed`	Server -> Client	Task execution succeeds
`task_failed`	Server -> Client	Task execution fails
`tool_called`	Server -> Client	Tool invocation in ReAct loop
`delegation_started`	Server -> Client	Agent delegation begins
`delegation_completed`	Server -> Client	Agent delegation succeeds
`delegation_failed`	Server -> Client	Agent delegation fails
`review_requested`	Server -> Client	Review gate awaiting decision
`review_timed_out`	Server -> Client	Review gate timeout expired
`review_decision`	Client -> Server	Human review decision
`token`	Server -> Client	Streaming LLM token
`heartbeat`	Server -> Client	Connection keepalive
`ping`	Client -> Server	Client keepalive
`pong`	Server -> Client	Ping response

New v3.0.0 Messages: Registration

`ensemble_register`

Sent by an ensemble when it joins the network, publishing its shared capabilities.

```
{
  "type": "ensemble_register",
  "name": "kitchen",
```

```
  "realm": "hotel-downtown",
  "capabilities": {
    "sharedTasks": [
      {
        "name": "prepare-meal",
        "description": "Prepare a meal as specified",
        "tags": ["food", "kitchen"]
      }
    ],
    "sharedTools": [
      {
        "name": "check-inventory",
        "description": "Check ingredient availability",
        "tags": ["food", "inventory"]
      }
    ]
  }
}
```

ensemble_deregister

Sent when an ensemble is shutting down.

```
{
  "type": "ensemble_deregister",
  "name": "kitchen",
  "realm": "hotel-downtown"
}
```

capacity_update

Periodic capacity advertisement (default: every 30 seconds).

```
{
  "type": "capacity_update",
  "ensemble": "kitchen",
  "realm": "hotel-downtown",
  "status": "available",
  "currentLoad": 0.65,
  "maxConcurrent": 10,
  "availableCapacity": 3,
  "queueDepth": 7,
  "shareable": true
}
```

New v3.0.0 Messages: Work Requests

task_request

WorkRequest envelope for cross-ensemble task delegation.

```
{
  "type": "task_request",
  "requestId": "maint-7721-valve-order",
  "from": "maintenance",
  "task": "purchase-parts",
  "context": "Order replacement valve model X-420 for building 2 boiler, urgent priority",
  "priority": "HIGH",
  "deadline": "PT30M",
  "delivery": {
    "method": "QUEUE",
    "address": "maintenance.results"
  },
  "traceContext": {
    "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
    "tracestate": "agentensemble=maintenance"
  },
  "cachePolicy": "FORCE_FRESH",
  "cacheKey": null
}
```

task_accepted

Acknowledgment with queue position and estimated completion time.

```
{
  "type": "task_accepted",
  "requestId": "maint-7721-valve-order",
  "queuePosition": 3,
  "estimatedCompletion": "PT15M"
}
```

task_progress

Optional streaming progress update during task execution.

```
{
  "type": "task_progress",
  "requestId": "maint-7721-valve-order",
  "status": "IN_PROGRESS",
  "message": "Searching vendors for model X-420...",
  "percentComplete": 30
}
```

`task_response`

Work result (completed, failed, or rejected).

```
{
  "type": "task_response",
  "requestId": "maint-7721-valve-order",
  "status": "COMPLETED",
  "result": "Ordered from SupplyCo, PO #4821, estimated delivery Thursday.",
  "durationMs": 45000,
  "tokenCount": 2841
}
```

Status values: `COMPLETED`, `FAILED`, `REJECTED`.

Failed response:

```
{
  "type": "task_response",
  "requestId": "maint-7722",
  "status": "FAILED",
  "error": "No vendors found for model X-420 in the approved supplier list."
}
```

Rejected response (at hard capacity limit):

```
{
  "type": "task_response",
  "requestId": "maint-7723",
  "status": "REJECTED",
  "reason": "AT_CAPACITY",
  "currentQueue": 200,
  "suggestion": "Try kitchen in realm hotel-airport"
}
```

`tool_request`

Request to execute a shared tool on a remote ensemble.

```
{
  "type": "tool_request",
  "requestId": "rs-8801",
  "from": "room-service",
  "tool": "check-inventory",
  "input": "wagyu beef",
  "traceContext": {
    "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-11a2b3c4d5e6f7a8-01"
  }
}
```

tool_response

Result of a shared tool execution.

```
{
  "type": "tool_response",
  "requestId": "rs-8801",
  "status": "COMPLETED",
  "result": "Yes, 3 portions available",
  "durationMs": 120
}
```

New v3.0.0 Messages: Human Interaction

directive

Non-blocking guidance from a human or automated policy.

```
{
  "type": "directive",
  "to": "room-service",
  "from": "manager:human",
  "content": "Guest in 801 is VIP, prioritize all their requests",
  "ttl": "P3D"
}
```

Control plane directive:

```
{
  "type": "directive",
  "to": "kitchen",
  "from": "cost-policy:automated",
  "action": "SET_MODEL_TIER",
  "value": "FALLBACK"
}
```

query

Request information from an ensemble.

```
{
  "type": "query",
  "queryId": "q-4401",
  "to": "front-desk",
  "from": "manager:human",
  "question": "How many rooms are available tonight?"
}
```

query_response

Response to a query.

```
{
  "type": "query_response",
  "queryId": "q-4401",
  "answer": "47 rooms available: 12 standard, 28 deluxe, 7 suites."
}
```

notification

Alert from an ensemble to a human.

```
{
  "type": "notification",
  "from": "maintenance",
  "severity": "HIGH",
  "message": "Water leak detected in room 305, emergency response dispatched.",
  "targetRole": "manager"
}
```

New v3.0.0 Messages: Network Coordination

profile_applied

Broadcast when an operational profile is applied to the network.

```
{
  "type": "profile_applied",
  "profile": "sporting-event-weekend",
  "appliedBy": "manager:human",
  "appliedAt": "2026-03-06T16:00:00Z"
}
```

capability_query

Discovery query: who provides this capability?

```
{
  "type": "capability_query",
  "queryId": "cq-001",
  "from": "room-service",
  "capability": "check-inventory",
  "tags": ["food"]
}
```

capability_response

Discovery response: list of providers.

```
{
  "type": "capability_response",
  "queryId": "cq-001",
```

```
  "providers": [
    {
      "ensemble": "kitchen",
      "realm": "hotel-downtown",
      "type": "TOOL",
      "healthy": true,
      "currentLoad": 0.65
    },
    {
      "ensemble": "kitchen",
      "realm": "hotel-airport",
      "type": "TOOL",
      "healthy": true,
      "currentLoad": 0.20
    }
  ]
}
```

Appendix B: SPI Reference

The Ensemble Network defines pluggable interfaces (SPIs) for all infrastructure-dependent components. Users implement these interfaces to integrate with their specific infrastructure. Default implementations are provided for development (in-memory) and production (Redis, Kafka).

Transport Layer

RequestQueue

Durable queue for work request delivery. Implementations must support consumer groups (multiple replicas reading from the same queue with at-least-once delivery).

```
public interface RequestQueue {

    /** Enqueue a work request for a target ensemble. */
    void enqueue(String queueName, WorkRequest request);

    /** Dequeue the next work request, blocking up to the given timeout. Returns null on timeo
    WorkRequest dequeue(String queueName, Duration timeout);

    /** Acknowledge successful processing. Removes the message from the queue. */
    void acknowledge(String queueName, String requestId);
}
```

Provided implementations: - `RequestQueue.inMemory()` – `ConcurrentLinkedQueue`, single-JVM, development only - `RequestQueue.redis(RedisClient)` – Redis Streams with consumer groups - `RequestQueue.kafka(Properties)` – Kafka consumer groups

ResultStore

Shared key-value store for work results. Used for the asymmetric routing pattern where the response is written by one pod and read by another.

```
public interface ResultStore {

    /** Store a work response keyed by request ID, with a TTL for automatic cleanup. */
    void store(String requestId, WorkResponse response, Duration ttl);

    /** Retrieve a stored response. Returns null if not found or expired. */
    WorkResponse retrieve(String requestId);
```

```
    /** Subscribe for notification when a result for the given request ID is stored. */
    void subscribe(String requestId, Consumer<WorkResponse> callback);
}
```

Provided implementations: - `ResultStore.inMemory()` – `ConcurrentHashMap`, single-JVM, development only - `ResultStore.redis(RedisClient)` – Redis with TTL and pub/sub notification

Transport

Factory combining RequestQueue and ResultStore into a transport configuration.

```
public interface Transport {

    /** Simple mode: in-process queues + direct WebSocket. No external infrastructure. */
    static Transport websocket() { ... }

    /** Durable mode: external queue + external result store. Production-grade. */
    static Transport durable(RequestQueue queue, ResultStore store) { ... }
}
```

Delivery

DeliveryHandler

Delivers work responses via the caller-specified delivery method.

```
public interface DeliveryHandler {

    /** Returns the delivery method this handler supports. */
    DeliveryMethod method();

    /** Deliver the response to the specified address. */
    void deliver(String address, WorkResponse response);
}
```

Provided implementations: - `WebSocketDeliveryHandler` – Direct WebSocket send - `QueueDeliveryHandler` – Enqueue to a durable queue (Redis Streams, SQS) - `TopicDeliveryHandler` – Publish to a topic (Kafka) - `WebhookDeliveryHandler` – HTTP POST to a URL - `StoreDeliveryHandler` – Write to the ResultStore - `BroadcastClaimDeliveryHandler` – Announce to all replicas; first to claim receives - `NoOpDeliveryHandler` – Fire and forget (NONE method)

Ingress

Normalizes work arriving from various sources into WorkRequest envelopes.

```
public interface Ingress {

    /** Start accepting work. Calls the handler for each received WorkRequest. */
    void start(Consumer<WorkRequest> handler);
```

```
    /** Stop accepting work. */
    void stop();
}
```

Provided implementations: - `WebSocketIngress` – Receives `task_request`/`tool_request` from WebSocket connections - `QueueIngress` – Pulls from a durable queue (Redis Streams, Kafka, SQS) - `HttpIngress` – Accepts `POST /api/work` HTTP requests - `TopicIngress` – Subscribes to a message topic

Result Caching

ResultCache

Optional result caching for repeated queries.

```
public interface ResultCache {

    /** Store a cached result with the given key and max age. */
    void cache(String cacheKey, WorkResponse response, Duration maxAge);

    /** Retrieve a cached result. Returns null if not found or expired. */
    WorkResponse get(String cacheKey);

    static ResultCache inMemory() { ... }
    static ResultCache redis(RedisClient client) { ... }
}
```

Shared State

LockProvider

Distributed locking for shared memory scopes with `Consistency.LOCKED`.

```
public interface LockProvider {

    /** Acquire a lock on the given scope. Blocks up to timeout. Returns a lock handle. */
    LockHandle acquire(String scope, Duration timeout);

    /** Release a previously acquired lock. */
    void release(LockHandle handle);

    static LockProvider inMemory() { ... }
    static LockProvider redis(RedisClient client) { ... }
}

public interface LockHandle extends AutoCloseable {
    String scope();
    void close();  // releases the lock
```

```
}
```

Observability

AuditSink

Receives audit records for storage or forwarding.

```
public interface AuditSink {

    /** Record an audit event. Must be non-blocking (implementations should buffer/batch). */
    void record(AuditRecord record);

    static AuditSink log() { ... }                    // SLF4J structured logging
    static AuditSink database(DataSource ds) { ... }  // JDBC append-only table
    static AuditSink eventStream() { ... }            // Kafka topic
}
```

ReviewNotifier

Sends out-of-band notifications when a gated review is pending and no qualified human is connected.

```
public interface ReviewNotifier {

    /** Send a notification that a review is pending. */
    void notify(ReviewNotification notification);

    static ReviewNotifier slack(String webhookUrl) { ... }
    static ReviewNotifier email(EmailConfig config) { ... }
    static ReviewNotifier webhook(String url) { ... }
}

public record ReviewNotification(
    String reviewId,
    String taskDescription,
    String requiredRole,
    String prompt,
    String dashboardUrl
) {}
```

Summary

SPI	Module	Purpose
`RequestQueue`	agentensemble-network	Durable work request delivery
`ResultStore`	agentensemble-network	Shared result storage
`Transport`	agentensemble-network	Transport mode factory
`DeliveryHandler`	agentensemble-network	Pluggable response delivery

SPI	Module	Purpose
`Ingress`	agentensemble-network	Pluggable work ingestion
`ResultCache`	agentensemble-network	Optional result caching
`LockProvider`	agentensemble-memory	Distributed locking
`AuditSink`	agentensemble-network	Audit record storage
`ReviewNotifier`	agentensemble-review	Out-of-band review notifications

Appendix C: Comparison Matrix

Feature-by-feature comparison of the Ensemble Network with existing multi-agent frameworks, the Model Context Protocol, and traditional microservice orchestration patterns.

Multi-Agent Framework Comparison

Capability	CrewAI	AutoGen	LangGraph	MCP	Ensemble Network
Execution Model					
Multi-agent orchestration	Yes	Yes	Yes	No	Yes
Long-running services	No	No	No	Yes (server)	Yes
One-shot execution	Yes	Yes	Yes	N/A	Yes
Event-driven / reactive	No	Partial	No	No	Yes
Scheduled / proactive tasks	No	No	No	No	Yes
Cross-Service Communication					
Cross-process communication	No	No	Cloud only	Yes (tools)	Yes (tasks + tools)
Task-level delegation	No	No	No	No	Yes
Tool-level sharing	No	No	No	Yes	Yes
Natural language contracts	N/A	N/A	N/A	Typed schemas	Yes

Capability	CrewAI	AutoGen	LangGraph	MCP	Ensemble Network
Async with priority queuing	No	No	No	No	Yes
Human Interaction					
Human-in-the-loop	Limited	Yes (chat)	Limited	No	Full spectrum
Non-blocking directives	No	No	No	No	Yes
Role-based gated reviews	No	No	No	No	Yes
Human-optional operation	No	No	No	N/A	Yes
Late-join state catch-up	No	No	No	No	Yes
Infrastructure					
Durable work delivery	No	No	No	No	Yes (queue + store)
Distributed tracing	No	No	No	No	OTel integration
Elastic scaling (K8s HPA)	No	No	No	No	Yes
Graceful shutdown / drain	No	No	No	No	Yes
Operational profiles	No	No	No	No	Yes
Resilience					
Timeouts	Basic	Basic	Basic	Basic	Per-call configurable
Retry policies	No	No	No	No	Yes (transient vs business)
Circuit breakers	No	No	No	No	Yes
Fallback strategies	No	No	No	No	Yes
Testing					
Test stubs for remote calls	No	No	No	No	Yes (built-in)
Simulation mode	No	No	No	No	Yes
Chaos engineering	No	No	No	No	Yes (built-in)

Capability	CrewAI	AutoGen	LangGraph	MCP	Ensemble Network
Contract testing support	No	No	No	No	Yes
State Management					
Cross-run memory	No	No	Checkpointing	No	Yes (scoped memory)
Cross-service shared state	No	No	No	Resources	Yes (configurable consistency)
Result caching	No	No	No	No	Yes (pluggable)
Observability					
Execution traces	Limited	Limited	Limited	No	Full (ExecutionTrace)
Adaptive audit trail	No	No	No	No	Yes (dynamic rules)
Scaling metrics	No	No	No	No	Yes (Micrometer)
Live dashboard	No	No	No	No	Yes (agentensemble-viz)
Federation					
Cross-cluster capability sharing	No	No	No	No	Yes
Capacity advertisement	No	No	No	No	Yes
Load-based routing	No	No	No	No	Yes

Microservice Pattern Comparison

The Ensemble Network borrows established patterns from microservice architecture and adapts them for AI agent systems. The key difference is the contract model: microservices use typed schemas; the Ensemble Network uses natural language.

Pattern	Traditional Microservices	Ensemble Network
Service discovery	Consul, K8s DNS, Eureka	K8s DNS + capability registration
API contract	OpenAPI / JSON Schema	Natural language description
Schema versioning	Semantic versioning, deprecation	Not needed (LLM compatibility)
Circuit breaker	Hystrix, Resilience4j	Built-in, per-target

Pattern	Traditional Microservices	Ensemble Network
Message queue	RabbitMQ, Kafka, SQS	Same (pluggable SPI)
Distributed tracing	OpenTelemetry, Jaeger	Same (OTel integration)
Health checks	K8s probes	Same (framework provides endpoints)
Auto-scaling	K8s HPA	Same (framework provides metrics)
Graceful shutdown	SIGTERM + drain	Same (lifecycle state machine)
Load balancing	K8s Service, Envoy	Same (K8s Service)
Rate limiting	Envoy, API gateway	Priority queue + capacity limits
Authentication	mTLS, OAuth, API keys	Same (SPI, delegates to infra)

MCP Detailed Comparison

MCP and the Ensemble Network operate at different levels of abstraction. They are complementary, not competing.

Aspect	MCP	Ensemble Network
What is shared	Functions (tools) and data (resources)	Tasks (multi-step workflows) and tools
Execution complexity	Stateless function call	Stateful multi-step process with agents, tools, memory, review gates
Caller control	Caller invokes function, gets result	Caller delegates; provider runs full process autonomously
Human involvement	Not addressed	Full spectrum (autonomous to gated)
Async support	Synchronous	Async with priority queue, multiple request modes
State	Stateless (tools) or read-only (resources)	Shared memory with configurable consistency
Transport	stdio, SSE, HTTP	WebSocket + durable queues (pluggable)
Discovery	Static (configuration)	Static + dynamic (capability registration + catalog)
Resilience	Caller retries	Timeouts, retries, circuit breakers, fallbacks, durable queues
Deployment model	Client-server	Peer-to-peer mesh
Scaling	Not addressed	K8s-native with HPA
Contract evolution	Schema versioning	Natural language (no versioning needed)

Complementary Usage

An ensemble could: - **Expose its shared tools as an MCP server**: External MCP clients (IDE plugins, other frameworks) can discover and call the ensemble's tools via MCP. - **Consume external MCP servers**: An ensemble's agents can use tools hosted by MCP servers (database tools, code analysis tools, etc.) alongside local and network tools.

The integration point is the tool interface. Both MCP tools and Ensemble Network tools implement the same `AgentTool` abstraction. An agent sees a flat list of available tools and does not distinguish between local tools, MCP tools, network tools, and network tasks.